OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 14

DEVELOPING INFORMATION SYSTEMS

SUGGESTED TIME: 4 PERIODS

IN THIS CHAPTER

This chapter presents the complete life cycle of an information system, from initial analysis to daily management. Students learn the development process, master design tools such as DFDs and flowcharts, apply rigorous testing strategies, and understand how information systems are managed and maintained in real organisations.

- By the end of this chapter I can describe the stages of the system development life cycle (SDLC) and explain the purpose of each stage.
- By the end of this chapter I can compare development models (waterfall, V-model, prototyping, iterative/incremental, agile) and choose an appropriate one for a given project.
- By the end of this chapter I can distinguish analysis from design and identify the deliverables produced at each phase.
- By the end of this chapter I can construct and interpret system design tools: data flow diagrams, flowcharts, structure charts, decision tables and data dictionaries.
- By the end of this chapter I can design an interface and a simple database schema consistent with a requirements specification.
- By the end of this chapter I can explain the levels of testing (unit, integration, system, acceptance) and design test cases with expected results.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. The System Development Process
2. System Design Tools
3. System Testing and Implementation
4. Information System Management
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can describe the stages of the system development life cycle (SDLC) and explain the purpose of each stage.
- By the end of this chapter I can compare development models (waterfall, V-model, prototyping, iterative/incremental, agile) and choose an appropriate one for a given project.
- By the end of this chapter I can distinguish analysis from design and identify the deliverables produced at each phase.
- By the end of this chapter I can construct and interpret system design tools: data flow diagrams, flowcharts, structure charts, decision tables and data dictionaries.
- By the end of this chapter I can design an interface and a simple database schema consistent with a requirements specification.
- By the end of this chapter I can explain the levels of testing (unit, integration, system, acceptance) and design test cases with expected results.
- By the end of this chapter I can differentiate verification from validation and describe alpha, beta and user acceptance testing.
- By the end of this chapter I can compare implementation/conversion strategies (direct, parallel, phased, pilot) and justify a choice.
- By the end of this chapter I can describe the roles involved in managing an information system (CIO, DBA, analysts, users) and the main maintenance types.
- By the end of this chapter I can evaluate the success of an information system and propose improvements for its security, documentation and training.

PREREQUISITES

- Notion of an information system: hardware, software, data, procedures and users (Lower Sixth)
- Basic algorithmics: flowcharts, sequence, selection and iteration (Lower Sixth)
- Introduction to databases: tables, records, fields and primary keys (Lower Sixth)
- Programming fundamentals and the idea of debugging a program (Lower Sixth)
- General ICT vocabulary: hardware, software, networks and users' needs

1 THE SYSTEM DEVELOPMENT PROCESS

In 2023, a microfinance cooperative in Bamenda spent millions of FCFA on a new member-management software — only to abandon it six months later because tellers found it unusable and records were lost during migration. Meanwhile, the MINESEC examination results platform and the MTN Mobile Money system handle millions of Cameroonians' data daily with remarkable reliability. What makes the difference between an information system that transforms an organisation and one that collapses? The answer lies not in the technology itself, but in how the system was developed, tested and managed. This chapter takes you inside the disciplined process that turns an idea into a dependable information system.

The central problem of this section is therefore: **how do we move from a need expressed by an organisation to a working information system, without wasting money, time and trust?**

1.1 Information Systems and Why They Fail

DEFINITION **Information system**

An **information system** (IS) is an organised set of people, procedures, hardware, software and data that collects, stores, processes and distributes information to support the operations and decision-making of an organisation.

Notice that an information system is not just software. The MTN Mobile Money system includes the application code, but also the agents, the SIM cards, the network, the transaction databases and the procedures for reversing an erroneous transfer. Forgetting the human and procedural parts is one of the most common causes of failure.

DEFINITION **System development**

System development is the structured process of planning, analysing, designing, building, testing, deploying and maintaining an information system so that it satisfies the needs of its users within given constraints of cost and time.

Why do systems fail? Experience in industry and in Cameroonian institutions points to three recurring reasons:

- **Cost overruns:** the project ends up costing far more than budgeted, so it is abandoned or delivered incomplete.
- **Time overruns:** the system arrives so late that the need has changed or the organisation has lost patience.
- **Unmet needs:** the delivered system does not do what users actually need — the classic case of software that is technically correct but practically unusable, like the Bamenda microfinance package.

All three causes share a common root: the absence of a disciplined process. The remedy is the System Development Life Cycle.

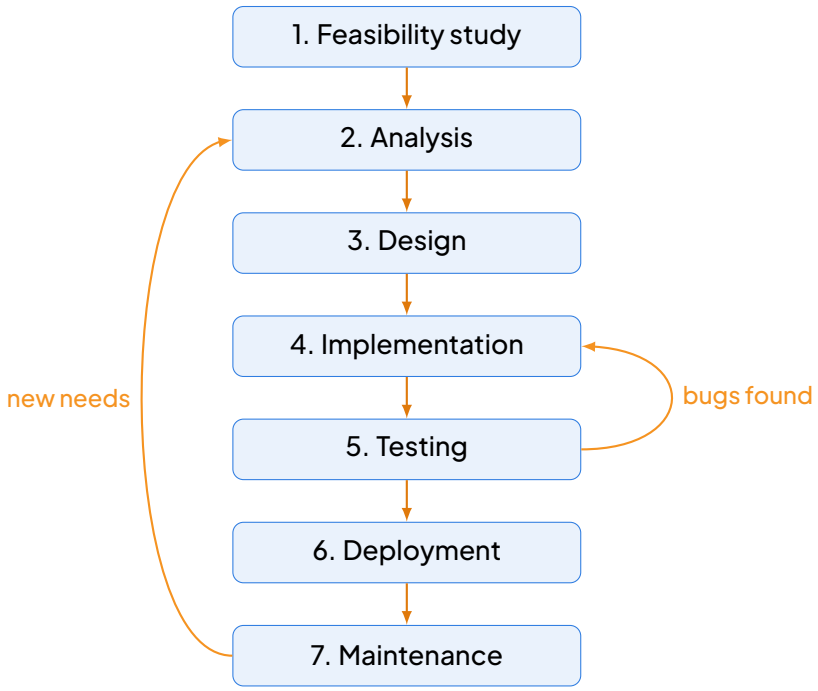
1.2 The System Development Life Cycle (SDLC)

DEFINITION **System Development Life Cycle**

The **System Development Life Cycle** (SDLC) is the structured sequence of phases through which an information system passes, from the first study of the problem to its retirement: preliminary investigation (feasibility study), analysis, design, implementation, testing, deployment and maintenance.

Let us describe each phase, using as a running example a school fees management system for a college in Buea.

1. **Preliminary investigation / feasibility study.** A problem or opportunity is identified ("fees records are kept in paper books and errors are frequent"). A short study determines whether the project is worth doing at all.
2. **Analysis.** The analyst studies the existing system in depth and establishes what the new system must do. The output is the requirements specification.
3. **Design.** The team decides how the system will do it: database structure, screen layouts, program modules, hardware and network choices.
4. **Implementation.** Programmers write the code, databases are created, and hardware is installed.
5. **Testing.** The system is checked systematically to find and correct errors before users depend on it.
6. **Deployment.** The new system is put into operation: data are migrated, users are trained, and the old system is replaced (directly, in parallel, or in phases).
7. **Maintenance.** Once in service, the system is corrected (bugs), adapted (new rules, e.g. new fee categories) and improved (new features) until it is eventually retired.



The SDLC stages, with feedback loops: testing can send the project back to implementation, and maintenance generates new needs that restart analysis.

KEY POINTS **Key idea**

The SDLC is a cycle, not a one-way street. Feedback loops are normal: errors discovered late are corrected by returning to earlier phases — and the later the discovery, the more expensive the correction.

DID YOU KNOW? **The cost of late changes**

Studies across the software industry consistently show that fixing a defect found during maintenance costs 10 to 100 times more than fixing the same defect during design. This is why the SDLC emphasises getting requirements and design right early — a principle that applies equally to a school fees system in Buea and to the core banking software of a national bank.

1.3 The Feasibility Study: the TELOS Checklist

Before investing a single franc in development, the organisation must answer: should we do this project at all? The feasibility study examines five dimensions, remembered by the acronym **TELOS**.

METHOD **Conducting a feasibility study with TELOS**

1. Technical feasibility: do we have (or can we obtain) the technology, equipment and technical skills? Example: does the college have computers and a reliable power supply?
2. Economic feasibility: do the expected benefits exceed the costs of development and operation? Compare hardware, licences, salaries and training against savings and new revenue.
3. Operational feasibility: will the system actually be used? Will staff accept it? Does it fit the organisation's procedures and culture?
4. Legal feasibility: does the project comply with laws and regulations (data protection, contracts, licences)?
5. Schedule feasibility: can the system be delivered within the required time? Example: before the start of the next school year.

If any dimension fails seriously, the project must be redefined, postponed or abandoned.

EXEMPLE **TELOS applied to the Bamenda microfinance**

A quick TELOS review would have revealed: operational risk (tellers not consulted, low computer literacy), schedule risk (migration planned in one weekend), and economic risk (no budget for training). The project was technically possible but operationally doomed — exactly the failure that occurred.

1.4 Analysis: Fact-Finding and Requirements

Once the project is approved, the **systems analyst** studies the existing situation. Four classic **fact-finding techniques** are used:

- **Interviews:** face-to-face discussions with managers and users. Rich, detailed answers, but time-consuming and dependent on the interviewer's skill.
- **Questionnaires:** written questions distributed to many users. Cheap and fast for large groups, but answers are shallow and the response rate may be low.
- **Observation:** the analyst watches users actually working. Reveals what people really do (often different from what they say), but people may change their behaviour when observed.
- **Document review:** studying existing forms, reports, records and procedure manuals (e.g. fee receipts, report cards). Gives precise facts about data, but documents may be outdated.

A good analyst combines several techniques: documents give the facts, interviews give the

reasons, observation gives the truth.

The result of analysis is the **requirements specification**, which distinguishes two kinds of requirements.

DEFINITION **Functional and non-functional requirements**

A **functional requirement** states what the system must do — a service or behaviour. A **non-functional requirement** states how well the system must do it — a quality or constraint (performance, security, usability, reliability).

EXEMPLE **Requirements for the college fees system**

Functional: "The system shall record a fee payment and issue a printed receipt." "The system shall produce the list of students who have not paid, class by class."
Non-functional: "The system shall print a receipt in under 3 seconds." "Only the bursar shall modify payment records." "The system shall be usable in English and French."

All requirements are gathered in a formal document, the **Software Requirements Specification** (SRS), which serves as the contract between the users and the developers: it is the reference against which the final system will be accepted.

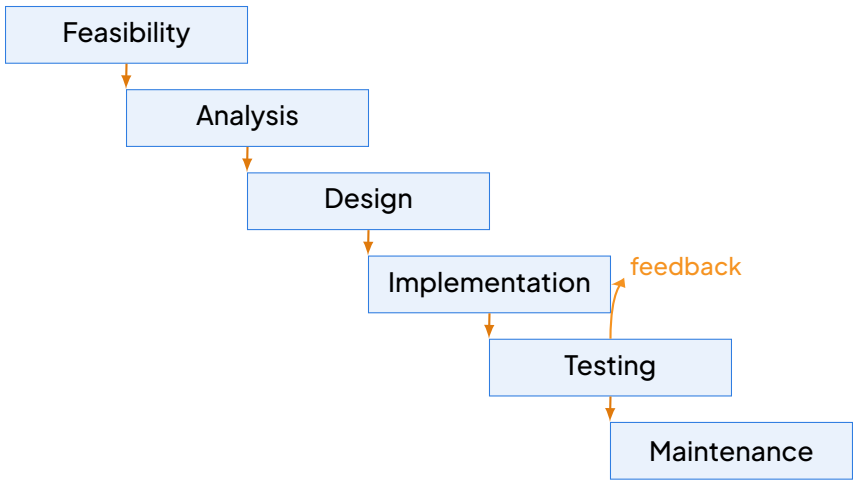
WARNING **Analysis vs design — a classic exam trap**

Do not confuse the analysis phase, which defines what the system must do ("record payments, list debtors"), with the design phase, which defines how the system will do it ("a MySQL database with a `payments` table, a receipt screen with three buttons"). Writing database tables during analysis, or listing user needs during design, is a category error that costs marks at the GCE.

1.5 Development Models

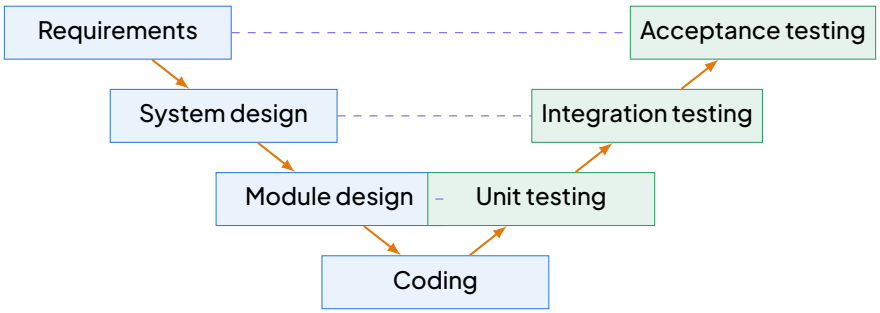
The SDLC tells us which phases exist; a **development model** tells us how to organise them in time.

The waterfall model. Phases follow one another strictly, each completed and documented before the next begins, like water falling down steps.



The waterfall model: each phase ends before the next starts; feedback to the previous phase is possible but costly.

The V-model. A variant of the waterfall in which every development phase on the left has a corresponding test level on the right: each specification document becomes the basis of a test plan written early.



The V-model: each development phase (left) is validated by a matching test level (right); dashed links show which specification feeds which test.

Prototyping. A quick, incomplete working model (a **prototype**) of the system is built and shown to users, whose feedback leads to improved versions, until the final system is agreed. Ideal when users cannot express their needs precisely in advance.

Iterative / incremental model. The system is built and delivered in small pieces (increments): version 1 handles payments only, version 2 adds reports, version 3 adds SMS notifications. Each increment passes through the full cycle.

Spiral model. The project turns through repeated cycles of plan – analyse risks – build – evaluate, the "radius" growing each turn. Designed for large, high-risk projects where risk analysis drives each iteration.

Agile methods. Short development cycles ("sprints" of a few weeks) deliver working software

continuously, with users involved at every step and changing requirements welcomed rather than feared.

PROPERTY **Waterfall vs prototyping**

The waterfall model is rigid and document-driven: it works only when requirements are clear, stable and fully known in advance. Prototyping reduces the risk of user rejection, because users see and try the system early and misunderstandings are corrected while they are still cheap. (Not examinable as a proof, but the comparison itself is a frequent exam question.)

Criterion	Waterfall	Prototyping
Requirements	Fixed and complete at the start	Discovered progressively with users
User involvement	Mainly at beginning and end	Continuous, throughout development
Working software visible	Only near the end	Very early (first prototype)
Cost of changing requirements	Very high	Low
Documentation	Heavy, contract-like	Lighter; risk of poor final docs
Best for	Stable, well-understood projects	Unclear needs, user-interface-critical systems
Main risk	Delivering the wrong system late	Endless refinement, weak structure

METHOD **Choosing a development model**

1. Project size: small, well-defined project → waterfall; large, complex project → spiral or incremental.
2. Clarity of requirements: clear and stable → waterfall or V-model; vague or evolving → prototyping or agile.
3. Risk level: high technical or financial risk → spiral (risk analysis each cycle).
4. User availability: users available for frequent feedback → prototyping or agile; users unavailable → document-driven models.
5. Need for early delivery: partial results needed quickly → incremental or agile.

1.6 The Project Team and User Involvement

System development is teamwork. The **systems analyst** is the central figure: he or she inves-

tigates the existing system, gathers requirements, writes the SRS, coordinates between users and programmers, and follows the project through testing and deployment. Around the analyst work the **project manager** (planning, budget, deadlines), **programmers** (coding), **database administrators**, **testers** and, crucially, the **users** themselves.

User involvement is not a courtesy; it is a success factor. Users validate the requirements, react to prototypes, take part in acceptance testing and are trained before deployment. The Bamenda cooperative failed precisely because tellers — the real users — were never consulted until the system was imposed on them.

EXPERIMENT **Mini feasibility study**

Activity. In groups of three, choose a real or imagined organisation in your area (a school library, a village pharmacy, a taxi cooperative). Perform a rapid TELOS feasibility study for a simple information system to solve one of their problems. Present your findings in a 5-minute pitch: which dimensions are green, which are red, and what is your go/no-go recommendation?

WORKED EXAMPLE 1 **Choosing a model for a hospital project**

Statement. A regional hospital in Garoua wants a patient-record system. The director admits that staff "do not really know what they want", the budget is tight, and a first working version is needed within four months for the consultation service. Recommend a development model and justify it with two arguments.

SOLUTION

Solution. The appropriate choice is the prototyping model (possibly combined with incremental delivery).
Argument 1 — unclear requirements: since staff cannot specify their needs in advance, a prototype lets them see and react to a concrete system, correcting misunderstandings early and cheaply.
Argument 2 — risk of rejection and time pressure: showing an early working version to the consultation service within weeks guarantees user involvement and acceptance, and an increment covering consultations can be delivered within the four-month deadline, while the waterfall model would deliver nothing usable for a year and might be rejected on arrival.

EXERCISE 1 **Feasibility and fact-finding**

A teachers' cooperative in Yaoundé wants a loan-management system. (a) State the five TELOS feasibility dimensions and give one question the analyst should ask for each. (b) Choose two fact-finding techniques suitable for studying how loans are currently

granted, and justify each choice in one sentence.

SOLUTION **Exercise 1**

(a) Technical: do we have the computers, network and skills? Economic: will savings and better loan recovery exceed development costs? Operational: will the loan officers accept and actually use the system? Legal: does the system respect regulations on members' personal data? Schedule: can it be ready before the next general assembly? (b) Document review, to examine existing loan application forms and repayment registers, which give exact data and procedures; observation, to watch how loan officers actually process applications, revealing informal practices that documents and interviews would miss.

KEY POINTS **To remember**

An information system combines people, procedures, hardware, software and data. Systems fail through cost overruns, delays and unmet needs — all prevented by a disciplined SDLC: feasibility (TELOS), analysis (what), design (how), implementation, testing, deployment, maintenance. Requirements are functional (services) or non-functional (qualities), gathered in the SRS. The choice of model — waterfall, V-model, prototyping, incremental, spiral or agile — depends on project size, requirement clarity, risk and user availability, and the systems analyst keeps users at the heart of the whole process.

2 SYSTEM DESIGN TOOLS

In the previous section we followed the life of an information system from the first feasibility study to its maintenance. Between analysis ("what must the system do?") and implementation ("let us write the code") lies a crucial stage: design. Design answers the question "**how** will the system do it?". Because a system is too complex to be held in one person's head, analysts use **design tools**: standardised diagrams and documents that model the system before any code is written.

Why model first? Three reasons. First, a diagram is far cheaper to correct than a program: moving a box on paper takes seconds, rewriting a module takes weeks. Second, diagrams are a **common language**: the analyst, the programmer and the future user (say, the bursar of a college in Bamenda) can all look at the same picture and agree on what the system will do. Third, models document the system for future maintenance teams. In this section we study the principal design tools required by the syllabus: data flow diagrams, flowcharts, structure charts, decision tables and trees, the data dictionary, input/output design, and a reminder of database design.

2.1 Data Flow Diagrams (DFD)

Imagine you want to computerise the fee-payment system of your school. Before drawing screens or writing code, you ask: where does data come from, what happens to it, where is it kept, and where does it go? A Data Flow Diagram answers exactly these questions, without saying how the processing is done (no loops, no decisions, no programming details).

DEFINITION **Data Flow Diagram**

A **Data Flow Diagram** (DFD) is a graphical model that shows how data moves through a system: which external entities supply or receive data, which processes transform it, and which data stores hold it. It uses exactly four symbols:

- External entity (rectangle): a person, organisation or other system outside the system being modelled, which sends data to it or receives data from it (e.g. Student, Bank).
- Process (circle or rounded rectangle, numbered): an activity that transforms incoming data flows into outgoing data flows (e.g. 1. Record payment). A process must have at least one input and one output.
- Data store (open-ended rectangle, labelled D1, D2, …): a place where data is kept (a file, a database table, a register). Data stores represent data at rest.
- Data flow (labelled arrow): data in motion between entities, processes and stores. Every arrow must be named with the data it carries.

A DFD is built in levels. The **context diagram** (often called Level 0) shows the whole system as one single process surrounded by its external entities: it fixes the boundary of the system. The **level-1 DFD** (first explosion) then "explodes" that single process into the main sub-processes (numbered 1, 2, 3, …) and reveals the data stores. If a process is still complex, it is exploded again into a level-2 DFD (processes 2.1, 2.2, …), and so on. **Balancing** must be preserved: the data flows crossing the boundary of a process at one level must match exactly the flows crossing the boundary of its exploded diagram at the next level.

METHOD **Drawing a DFD**

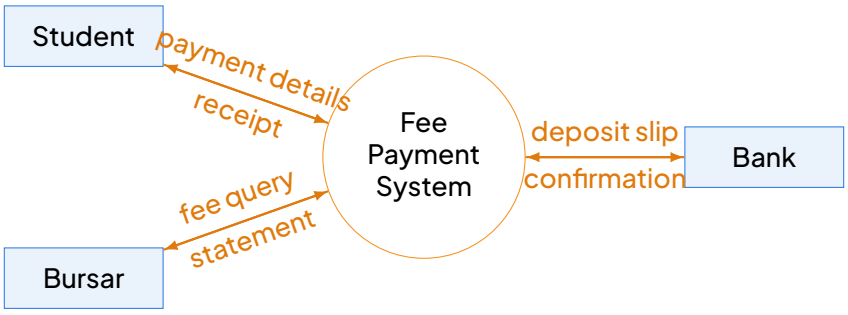
1. Identify the external entities: who or what interacts with the system from outside?
2. Identify the processes: what transformations does the system perform? Number them meaningfully.
3. Identify the data stores: what data must be remembered between processes?
4. Draw the data flows: connect entities, processes and stores with named arrows.
5. Check balancing: the flows entering and leaving a process at one level must correspond exactly to the flows entering and leaving its exploded sub-diagram at the next level. The context diagram and the level-1 DFD must show the same external flows.

WARNING **Illegal connections in a DFD**

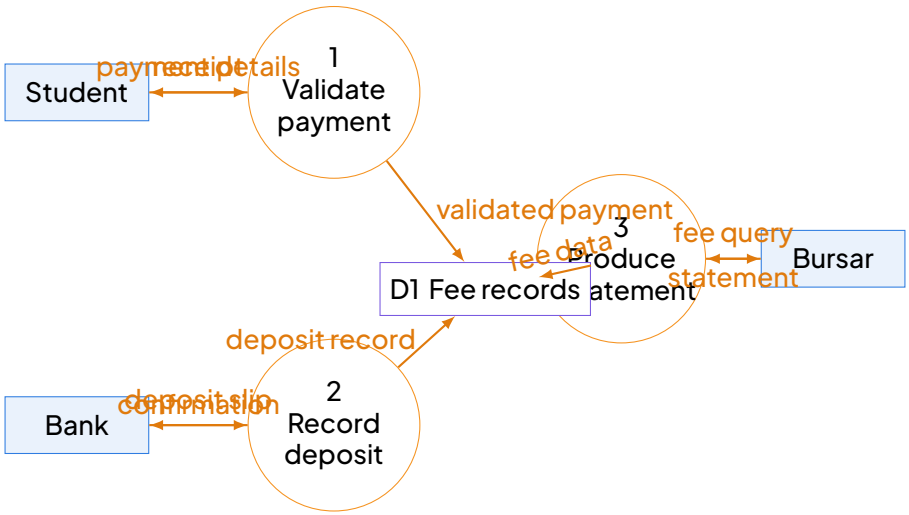
A data flow can never connect two data stores directly, nor two external entities directly. Data can only be moved or transformed by a process. So D1 → D2 is wrong; D1 → Process → D2 is correct. Likewise, every process must have at least one input flow and at least one output flow (a process with only inputs is a "black hole"; with only outputs, a "miracle"). These are classic traps in GCE Paper 2.

EXEMPLE **Fee-payment system: context diagram and level-1 DFD**

The context diagram shows the system Fee Payment System exchanging data with the Student (payment details in, receipt out), the Bursar (fee queries in, statements out) and the Bank (deposit slips in, confirmations out). The level-1 DFD explodes it into three processes and one data store.



Context diagram (Level 0) of the school fee-payment system: one process, three external entities.



Level-1 DFD: the system is exploded into three numbered processes and data store D1.

KEY POINTS **Balancing rule**

The external flows of the context diagram (payment details, receipt, deposit slip, confirmation, fee query, statement) must appear exactly on the level-1 DFD — no more, no fewer. Examiners check this first.

2.2 Flowcharts and System Flowcharts

A DFD shows what happens to data but not in what order steps are executed, nor where decisions are taken. For that we use a **flowchart**: a diagram of the logic of a procedure or algorithm, using standard symbols connected by arrows showing the order of execution.

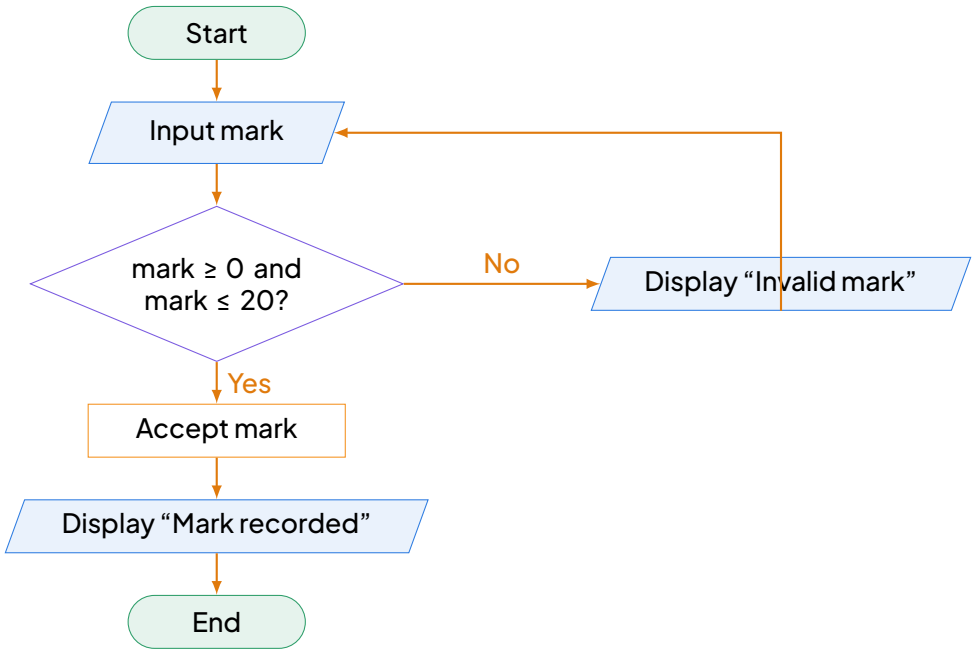
Symbol	Name	Meaning
Oval / rounded rectangle	Terminator	Start or End of the procedure
Parallelogram	Input / Output	Reading data in or displaying/printing data
Rectangle	Process	A computation or action (e.g. *total* ← *total* + *amount*)
Diamond	Decision	A test with two exits (Yes/No)
Small circle	Connector	Joins parts of a flowchart across a page
Arrow	Flow line	Direction of execution

A **system flowchart** uses the same idea at a higher level: it shows the flow of documents and data between programs, files and manual operations in the whole system (with extra symbols for documents, magnetic disk, manual input, etc.), whereas a **program flowchart** details the logic inside one program. Common system flowchart symbols include: document (wavy bottom), magnetic disk (cylinder), manual operation (trapezoid with flat top), and display (delay

symbol).

WARNING **Symbol mix-ups**

The three most confused symbols in examinations: rectangle = process, diamond = decision, parallelogram = input/output. Never put a calculation in a diamond, and never read data in a rectangle. A decision diamond must have exactly two labelled exits (Yes/No or True/False).



Program flowchart of a mark-validation procedure (marks out of 20), using the standard symbols.

2.3 Structure Charts: Modular Design

Large systems are designed **top-down**: the whole problem is one module, which is decomposed into sub-modules, each decomposed again, until every module is small enough to be programmed and tested by one person. A **structure chart** is the tree diagram that shows this hierarchy of modules and which module calls which. It also shows the **data couples** (arrows with empty circles) and **control couples** (arrows with filled circles) passed between modules.

DEFINITION **Coupling and cohesion**

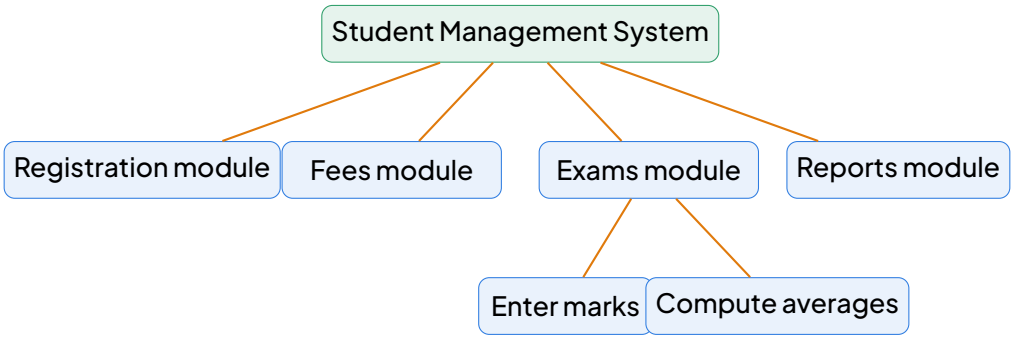
- **Coupling** measures how strongly two modules depend on each other (how much data and control they share). Types range from content coupling (worst) to data coupling

(best).

- **Cohesion** measures how strongly the elements inside one module belong together (does the module do one single, well-defined job?). Types range from coincidental cohesion (worst) to functional cohesion (best).

PROPERTY **Quality of a modular design**

A good modular design has high cohesion inside each module and low coupling between modules. High cohesion makes a module easy to understand, test and reuse; low coupling means a change in one module does not force changes in the others. (Statement to know for the examination; no formal proof is required.)



Structure chart of a student management system: top-down decomposition into cohesive modules.

2.4 Decision Tables and Decision Trees

Some business rules are too tangled for prose: "a loan is granted if the applicant is a member and has regular income, unless the amount exceeds three times the savings, in which case a guarantor is required…" A **decision table** makes such rules exhaustive and unambiguous: it lists all **conditions**, all **actions**, and one column per **rule** (combination of condition values). We use limited-entry tables where conditions are True/False (Y/N) and actions are marked with X.

METHOD **Building a decision table**

1. List the conditions (questions with Yes/No answers) in the upper-left part.
2. List the possible actions in the lower-left part.
3. Enumerate the rules: one column for each relevant combination of condition val-

ues (2^n columns for n independent conditions, then remove impossible or redundant ones).

4. For each rule, tick (X) the action(s) to perform.
5. Check completeness (every case covered) and consistency (no two rules give contradictory actions for the same case).

EXEMPLE **Loan approval in a credit union**

A credit union in Buea grants a loan under these rules: the applicant must be a registered member (C1) and have a regular income (C2); if the loan exceeds three times the applicant's savings (C3), a guarantor is required. The decision table is:

Conditions / Rules	R1	R2	R3	R4
C1: Registered member?	Y	Y	Y	N
C2: Regular income?	Y	Y	N	–
C3: Loan > 3× savings?	N	Y	–	–
Actions				
Grant loan	X			
Grant loan with guarantor		X		
Refuse loan			X	X

A decision tree represents the same logic as a branching diagram (conditions on the branches, actions at the leaves); it is easier to read for non-specialists but grows large quickly. The tree for this example would have three levels of branching (C1, then C2, then C3) with actions at the leaves.

2.5 The Data Dictionary

Diagrams name data flows and stores, but do not define them precisely. The **data dictionary** is the reference document that defines every data element of the system: its name, meaning, type, length, allowed values, validation rules, and origin. It ensures a common vocabulary between analysts, programmers, and users.

EXEMPLE **Extract from the fee-system data dictionary**

Element	Type	Length	Description / Validation / Origin
studentID	text	8	Unique code, e.g. STU00421; issued at registration; primary key
amountPaid	decimal	8.2	Amount in FCFA, > 0; from payment slip; presence and range check
paymentDate	date	10	Format DD/MM/YYYY; entered by bursar; must not be future date

2.6 Input/Output and Interface Design

A system succeeds only if people accept to use it. Design therefore covers:

- **Input design**: forms and screens that capture data at the source, with validation (presence check, format check, range check, check digit, consistency check) to keep errors out.
- **Output design**: reports and screens that give the right information, at the right time, in a readable layout (e.g. a termly fee-balance statement for parents). Consider media (screen, print, PDF), frequency, and distribution.
- **Interface (HCI) design**: the dialogue between user and machine.

KEY POINTS **Principles of good HCI**

- Consistency: same commands, colours and layouts everywhere in the system.
- Feedback: every action gets a visible response ("Payment saved").
- Error prevention and recovery: validate input, confirm destructive actions, allow undo, give clear error messages.
- Simplicity: minimise what the user must remember; use menus and defaults.
- User control: the user should feel in control, not the computer.
- Accessibility: cater for users with disabilities (colour blindness, motor impairments).

2.7 Database Design Overview (Reminder)

System design includes designing the permanent data stores. Recall from Lower Sixth:

- An **entity** is an object of interest (STUDENT, PAYMENT); it has **attributes** (name, date, amount); one attribute (or set) is the **primary key** that uniquely identifies each occurrence.
- An **Entity–Relationship (E-R) diagram** shows entities, their attributes and the relationships between them, with cardinalities (one-to-one, one-to-many, many-to-many).

- **Normalisation** (1NF, 2NF, 3NF) removes redundancy and update anomalies: atomic attributes (1NF), no partial dependency on part of a composite key (2NF), no transitive dependency (3NF).
- **Mapping to relational tables**: each entity becomes a table; each relationship becomes a foreign key (or a link table for many-to-many).

The data stores D1, D2, … of the DFD typically become the tables of the relational database.

2.8 Worked Examination-Style Exercise

WORKED EXAMPLE 2 **Completing a partially drawn DFD**

Statement. A library system has: external entities Borrower and Librarian; processes 1. Register loan and 2. Check availability; data store D1: Catalogue. The level-1 DFD is partially drawn: Borrower → (loan request) → Process 2; Process 2 → (availability status) → Borrower; Process 1 → (loan record) → D1. The following flows are missing: (a) Process 2 must read the catalogue; (b) the Librarian sends book details to Process 1; (c) Process 1 must confirm the loan to the Borrower. Draw/complete the diagram and state two rules of DFD construction you used.

SOLUTION

Solution.

- (a) Add a flow book details from D1 (Catalogue) to Process 2 (a store may be read by a process).
- (b) Add a flow book details from Librarian to Process 1 (entity → process is legal).
- (c) Add a flow loan confirmation from Process 1 to Borrower.

Rules used: (1) every flow must connect a process to an entity, a store or another process — never two stores or two entities directly; (2) every process must have at least one input and one output flow (here Process 1 receives book details and writes both the loan record and the confirmation). The diagram is balanced if these external flows match the context diagram.

EXERCISE 2 **Practice**

A hospital wants a DFD for its appointment system. Entities: Patient, Doctor. Stores: D1 Appointments file, D2 Patient records. Processes: 1. Book appointment, 2. Update patient record.

1. Draw the context diagram.
2. Draw the level-1 DFD with all flows named.

3. Explain why a direct flow from D1 to D2 would be wrong, and correct it.

SOLUTION **Exercise 1**

1. Context diagram: one circle Appointment System; Patient sends appointment request, receives confirmation; Doctor sends availability, receives daily schedule.
2. Level 1: Patient → (request) → 1. Book appointment; 1 → (confirmation) → Patient; 1 ↔ D1 (write booking / read slots); Doctor → (availability) → 1; 1 → (schedule) → Doctor; 2. Update patient record ↔ D2; 1 → (visit details) → 2.
3. D1 → D2 directly is illegal: data cannot move between two stores without a process. Correction: D1 → (appointment data) → 2. Update patient record → (updated record) → D2.

3 SYSTEM TESTING AND IMPLEMENTATION

Having designed our system architecture, databases, and user interfaces using the tools discussed in the previous section, we now reach a critical phase: bringing the design to life and proving it works. Writing code is only half the battle; the other half is ensuring that the code behaves exactly as the stakeholders expect, under all reasonable conditions. In the Cameroonian context, where software projects often face tight budgets and deadlines — think of a school management system for a bilingual college in Yaoundé or a mobile money integration for a startup in Douala — skipping thorough testing is a temptation that leads to catastrophic failures after deployment. This section covers the systematic approach to testing and the strategies for moving from the development environment to the live production environment.

3.1 The Purpose and Philosophy of Testing

Many students (and unfortunately, some practitioners) believe testing is about proving the system works. This is a dangerous misconception.

DEFINITION **Purpose of Testing**

Testing is the process of executing a system with the intent of finding faults (defects/bugs), not proving correctness. A successful test is one that discovers a previously unknown error.

KEY POINTS **Dijkstra's Principle**

"Testing can show the presence of bugs, but never their absence." — Edsger W. Dijkstra. No amount of testing can guarantee a system is 100% bug-free. It only increases confidence.

PROPERTY **Cost of Defect Discovery**

The cost to fix a defect increases exponentially the later it is found in the Software Development Life Cycle (SDLC).

Phase Found	Relative Cost to Fix
Requirements / Design	1× (Cheapest)
Coding / Unit Testing	5–10×
System / Integration Testing	20–50×
Post-Release (Production)	100× or more (Reputation, legal, data loss)

Examinable: Be able to explain why early testing (Static Testing, Reviews) saves money.

3.2 Verification vs. Validation

These two terms are constantly confused in examinations. The distinction is fundamental.

DEFINITION **Verification**

Verification: "Are we building the system right?"
It checks whether the products of a given phase (code, design, documents) meet the specifications set in the previous phase. It is static (reviews, walkthroughs, inspections, desk checking) and dynamic (testing against specs).

DEFINITION **Validation**

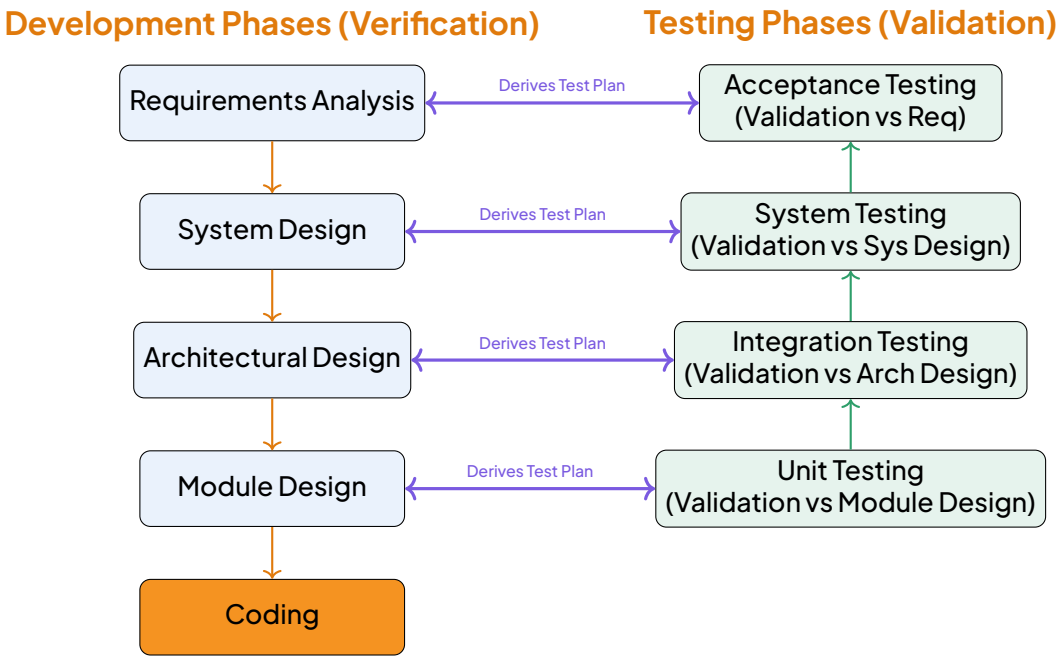
Validation: "Are we building the right system?"
It checks whether the final system meets the user's actual needs and requirements (User Requirements Specification). It answers: "Does this solve the user's problem?" It is primarily dynamic (User Acceptance Testing).

KEY POINTS **Mnemonic**

- V&V = Verification & Validation.
- Verification = Specifications match (Internal consistency). S for Specs.
- Validation = User needs met (External fitness). U for User.

3.3 Levels of Testing: The V-Model

Testing is not a single phase at the end; it maps directly to the development phases. The **V-Model** illustrates this relationship perfectly.



The V-Model: Linking Specification to Test

The V-Model: Each development phase on the left defines the test plan for the corresponding test level on the right.

3.4 Unit (Module) Testing

DEFINITION **Unit Testing**

Testing individual modules, functions, or classes in isolation. It is usually performed by the programmer who wrote the code (White-box testing).

- **Goal:** Verify internal logic, paths, loops, data structures.
- **Tools:** Stubs (simulate called modules) and Drivers (simulate calling modules).
- **Techniques:** Statement coverage, Branch/Decision coverage, Path coverage.

3.5 Integration Testing

Once units work alone, they must work together.

DEFINITION **Integration Testing**

Testing the interfaces and interactions between integrated modules.

Strategy	Description	Pros / Cons
Top-Down	Start with main module, integrate downwards using **Stubs** for lower modules.	+: Early demo of UI/Logic. -: Stubs complex; low-level utils tested late.
Bottom-Up	Start with lowest modules, integrate upwards using **Drivers**.	+: No stubs needed; core logic tested early. -: No working system until end; UI tested last.
Big Bang	All modules integrated at once.	-: Nightmare to debug; **Not recommended** for exams.
Sandwich (Hybrid)	Combines Top-Down and Bottom-Up.	+: Best of both worlds. Used in large systems.

3.6 System Testing

DEFINITION **System Testing**

Testing the complete, integrated system against the System Requirements Specification (SRS). Performed by an independent test team (not developers). It is Black-box testing.

3.7 Acceptance Testing (User Acceptance Testing – UAT)

DEFINITION **Acceptance Testing**

Formal testing conducted by the end-users/clients to determine whether to accept the system. It validates against User Requirements.

- **Alpha Testing:** Performed by **internal staff** (not developers) at the **developer's site**. Controlled environment.

- **Beta Testing:** Performed by **real end-users** at **their own sites** (real environment). Uncontrolled. Feedback on usability, compatibility, performance.

WARNING **Alpha vs Beta Trap**

Do not confuse them!

- Alpha = In-house (Developer's premises), Simulated/Controlled.
- Beta = External (User's premises), Real world, Limited release.

Exam questions often describe a scenario and ask you to identify which one it is.

3.8 Types of Testing (Non-Functional)

Beyond "does the button click?" (Functional), we test how it clicks.

Type	Focus
Performance / Load / Stress	Response time, throughput under normal load (Load), peak load (Stress), and breaking point.
Security	Vulnerabilities: SQL Injection, XSS, Authentication bypass, Data encryption, Access control. Critical for Mobile Money/Banking apps in Cameroon.
Usability	User experience (UX): Learnability, efficiency, accessibility (WCAG), error prevention.
Recovery / Reliability	System's ability to recover from crashes, hardware failure, power loss (common in regions with unstable electricity). Backup/Restore procedures.
Compatibility	Works on target OS (Windows/Linux/Mac), Browsers (Chrome/Firefox/Edge), Devices (Mobile/Desktop), Network conditions (3G/4G/WiFi).

3.9 Test Case Design: The Art of Finding Bugs

A **Test Case** is the atomic unit of testing.

DEFINITION **Test Case Structure**

A test case documents:

1. Test Case ID: Unique identifier (e.g., TC-LOGIN-001).
2. Input Data / Pre-conditions: Specific values, system state.
3. Test Steps: Actions to execute.
4. Expected Result: What the spec says should happen (Pass/Fail criteria).

5. Actual Result: What happened during execution.
6. Verdict: Pass / Fail / Blocked.

METHOD **Choosing Effective Test Data (Boundary Value Analysis & Equivalence Partitioning)**

To maximise fault detection with minimum test cases, apply this method for every input field:

1. Identify Equivalence Partitions: Divide input domain into classes where system behaviour should be identical (e.g., Valid Age: 18–60; Invalid: < 18, > 60, Negative, Text).
2. Select Representative Values:
   - One Normal (Valid) value: From the middle of a valid partition (e.g., Age = 25).
   - Boundary Values (Extreme): Edges of partitions (e.g., Age = 18, 60). Off-by-one errors live here. Test: Min, Min+, Max, Max-.
   - One Erroneous (Invalid) value: From an invalid partition (e.g., Age = 17, 61, -5, "twenty").
3. Document Expected Result for each.

EXEMPLE **Test Data for a "Student Age" Field (Range 11–18)**

Test Case	Input Data	Category / Expected Result
TC-01	14	Normal (Valid) → Accept
TC-02	11	Boundary (Min) → Accept
TC-03	10	Boundary (Min - 1) → Reject / Error Msg
TC-04	18	Boundary (Max) → Accept
TC-05	19	Boundary (Max + 1) → Reject / Error Msg
TC-06	-2	Erroneous (Negative) → Reject
TC-07	"Form 5"	Erroneous (Text) → Reject / Type Error

Note: Testing only 14 (Normal) would miss the boundary bugs at 11/18.

3.10 Black-Box vs White-Box Testing

Characteristic	Black-Box (Functional/Behavioural)	White-Box (Structural/Glass-Box)
Knowledge of Code	None (Internal structure hidden)	Full access to source code
Basis for Tests	Requirements Specification, User Stories	Code logic, Control flow, Data flow
Performed By	Independent Testers, End Users (UAT)	Developers (Unit Testing)
Techniques	Equivalence Partitioning, Boundary Value Analysis, Decision Tables, State Transition, Use Cases	Statement Coverage, Branch Coverage, Path Coverage, Cyclomatic Complexity
Finds	Missing functions, Interface errors, Data structure errors, Performance issues	Logic errors, Dead code, Infinite loops, Initialization errors

3.11 Dry Running / Desk Checking with Trace Tables

Before running code on a machine, the programmer simulates execution manually. This is **White-Box** static verification.

METHOD **How to Dry Run an Algorithm**

1. List all variables as column headers.
2. Add columns for Output and Condition/Line Number.
3. Execute line-by-line with chosen test data.
4. Update variable values only when they change.
5. Check logic against expected results.

WORKED EXAMPLE 3 **Dry Run: Finding the Maximum of 3 Numbers**

Algorithm:

1. Input A, B, C
2. Max ← A
3. If B > Max Then Max ← B
4. If C > Max Then Max ← C
5. Output Max

Test Data: A=10, B=25, C=15 (Normal), then A=5, B=5, C=5 (Boundary: All equal).
Solution:

Line	A	B	C	Max	Output / Condition
1 (Input)	10	25	15	?	
2	10	25	15	10	Max ← A
3	10	25	15	25	B(25) > Max(10)? Yes → Max=25
4	10	25	15	25	C(15) > Max(25)? No → No change
5	10	25	15	25	Output 25 ✓

Trace Table for Test Case 1 (Normal).

Line	A	B	C	Max	Output / Condition
1 (Input)	5	5	5	?	
2	5	5	5	5	Max ← A
3	5	5	5	5	B(5) > Max(5)? No (Strictly greater)
4	5	5	5	5	C(5) > Max(5)? No
5	5	5	5	5	Output 5 ✓

Trace Table for Test Case 2 (Boundary: All Equal).

Observation: The algorithm works for both cases. If the condition was >=, the trace table would show Max updating to B then C unnecessarily (logic inefficiency, though result correct).

3.12 Test Plan and Test Log

DEFINITION **Test Plan**

A document describing the scope, approach, resources, and schedule of testing activities. It includes: Features to test, Features not to test, Test strategy (levels, types), Pass/Fail criteria (Entry/Exit criteria), Test environment (Hardware/Software), Risks and contingencies, Schedule.

DEFINITION **Test Log (Test Execution Record)**

A chronological record of test execution. For each test case: Date, Tester, Test Case ID, Actual Result, Verdict (Pass/Fail), Defect ID (if Fail).

3.13 Implementation / Conversion Strategies

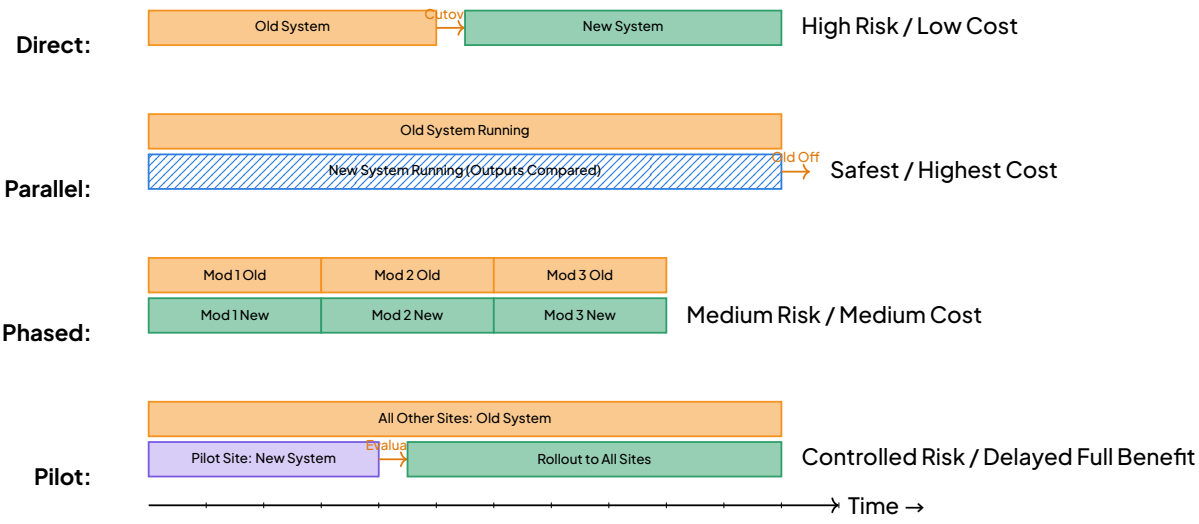
The system is tested and approved. Now we must move it to the live environment (**Go-Live**). This is **Changeover**.

PROPERTY **Conversion Strategies Comparison**

Strategy	Description	Advantages	Disadvantages / Risks
Direct Changeover (Big Bang)	Old system stopped immediately; New system starts immediately.	Cheapest; Fastest; No duplicate work.	Highest Risk. If new system fails → Total business halt. No fallback.
Parallel Running	Old and New systems run simultaneously for a period. Outputs compared.	Safest. Immediate fallback to old system. Results verified against live data. Staff trained gradually.	Most Expensive. Double workload (data entry twice), double cost, double staff effort. Data sync issues.
Phased (Modular) Conversion	System implemented module by module (e.g., Payroll first, then Inventory).	Risk spread out. Focus training/support per module. Fallback only for failed module.	Complex integration between old/new modules. Long transition period.
Pilot Conversion	Whole system implemented for a limited user group/location (e.g., One branch in Buea, or Form 5 class only).	Real-world test with limited exposure. Feedback before full rollout.	Pilot users bear burden. Other users wait. Not a full load test.

KEY POINTS **Choosing a Strategy – Exam Guidance**

- Direct: Small, non-critical systems; easy to revert manually; low cost tolerance.
- Parallel: Critical financial/medical systems (Banking, Hospital); high cost tolerance; legal requirement for audit trail.
- Phased: Large modular systems (ERP); distinct functional modules.
- Pilot: Geographically distributed orgs (Multi-branch bank, School chain); new technology stack.



Timeline comparison of the four implementation/changeover strategies.

3.14 Documentation Produced

A system is not complete until it is documented. GCE expects you to distinguish two main categories.

DEFINITION **User Documentation (User Manual)**

Target: End Users (Non-technical). Contents:

- Purpose of the system.
- Hardware/Software requirements (Minimum specs).
- Installation / Login guide.
- Step-by-step tutorials with screenshots for common tasks ("How to register a stu-

dent", "How to print a report card").

- Error messages meanings and user actions.
- FAQ / Troubleshooting.
- Glossary of terms.

DEFINITION **Technical Documentation (System Documentation)**

Target: IT Staff, Maintainers, Future Developers. Contents:

- System Architecture (Hardware/Network topology).
- Database Schema (ER Diagram, Data Dictionary, Table definitions, Triggers, Stored Procedures).
- Data Flow Diagrams (Context, Level 1).
- Program Specifications: Module hierarchy (Structure Chart), Pseudocode/Flowcharts for complex algorithms.
- Interface Specifications (API endpoints, File formats, Protocols).
- Test Plan, Test Cases, Test Logs, Test Results (Evidence of validation).
- Known Issues / Limitations / Future Enhancements.
- Security Configuration (Firewall rules, User roles/permissions matrix).

DEFINITION **Training Materials**

Distinct from the manual. Includes: Slide decks, Video tutorials, Quick Reference Cards (Cheat sheets), Sandbox/Training database environment.

3.15 Worked Examination Example

WORKED EXAMPLE 4 **GCE Style: Testing & Implementation Scenario**

Scenario: "A secondary school in Bamenda commissions a new 'Student Result Management System'. The old system was paper-based. The new system is a web application hosted on a school server. The school has 2,000 students. The bursar insists on verifying the calculated averages against manual calculations for the first term. The IT teacher

wants to test the 'Calculate Average' module logic thoroughly before integration."
Questions:

1. Identify the testing level the IT teacher is performing. Justify.
2. The bursar's request describes which testing level? Who performs it?
3. For the 'Calculate Average' module, the input is three marks (each 0–100). The output is the average (0–100). Design three test cases using Boundary Value Analysis.
4. The school decides to run the paper system and the web system simultaneously for the first term, comparing results. Name this conversion strategy and state one major disadvantage.
5. Distinguish between the User Manual and Technical Documentation for this system. Give one specific content example for each.

SOLUTION

Detailed Solution:
1. Unit Testing (Module Testing). Justification: The IT teacher is testing a single module ('Calculate Average') in isolation, focusing on internal logic, before it is integrated with other modules (e.g., Database, Report Generator). This is typically a White-box activity done by developers.
2. Acceptance Testing (User Acceptance Testing - UAT) / Parallel Running Verification. Who: The End Users (Bursar, Administration) or their representatives. Note: While this happens during Parallel Running, the act of verifying against user requirements is Acceptance Testing.
3. Test Cases for 'Calculate Average' (Inputs: Mark1, Mark2, Mark3; Range 0–100 each). Note: We test boundaries of the inputs and the output calculation.

TC ID	Mark 1	Mark 2	Mark 3	Category / Expected Output
TC-01	50	60	70	Normal (Mid-range). Avg = 60.0.
TC-02	0	100	50	Boundary (Min/Max inputs). Avg = 50.0. Tests Min (0) and Max (100) handling.
TC-03	-5	50	50	Erroneous (Invalid Input). System should reject / show validation error "Mark out of range".
(Optional TC-04)	100	100	100	Boundary (Max Output). Avg = 100.0.

4. Strategy: Parallel Running. Disadvantage: Double workload and cost. Staff must enter every mark twice (paper register + web app), doubling data entry time, stationery costs, and risk of transcription errors between the two systems. It is expensive and exhausting for teachers.
5. Documentation Distinction:

- User Manual: For teachers/bursar. Example content: “Step 1: Login using your staff ID. Step 2: Click ’Enter Marks’. Step 3: Select Class ’Form 5A’. Step 4: Type marks in grid. Step 5: Click ’Save & Calculate’. Screenshot of the marks entry screen.”
- Technical Documentation: For IT maintainers. Example content: “Database Table: `Student_Marks(StudentID PK, SubjectCode PK, Term, Mark1, Mark2, Mark3, Average, Grade)`. Stored Procedure: `sp_CalcAverage(StudentID, Term)` logic pseudocode. Server Specs: Ubuntu 22.04, Apache 2.4, MySQL 8.0, PHP 8.2. Backup Schedule: Daily 02:00 AM to /mnt/backup.”

3.16 Summary Checklist for Revision

KEY POINTS **Key Points for GCE CSC14 Section 3**

- Testing finds faults; it cannot prove correctness (Dijkstra).
- Verification (Right system? Specs) vs Validation (Right system? User needs).
- V-Model: Unit ↔ Module Design; Integration ↔ Arch Design; System ↔ Sys Design; Acceptance ↔ Requirements.
- Integration Strategies: Top-Down (Stubs), Bottom-Up (Drivers), Sandwich.
- Alpha (In-house) vs Beta (Real users, Real site).
- Test Data: Normal, Boundary (Min, Min+, Max, Max-), Erroneous. Boundary values catch off-by-one errors.
- Black-Box (Specs, User) vs White-Box (Code, Developer).
- Dry Run / Trace Table: Manual simulation, variable tracking.
- Conversion: Direct (Risky/Cheap), Parallel (Safe/Expensive), Phased (Modular), Pilot (Location/User group).
- Documentation: User Manual (How-to, Screenshots) vs Technical Doc (Schema, Code, Architecture, Test Evidence).

EXERCISE 3 **Practice Questions**

1. A programmer writes a function `CalculateDiscount(Amount, CustomerType)`. She creates a table of variable values and steps through the code line-by-line with sample inputs `Amount=10000, CustomerType='VIP'`. What is this process called? Is it Verification or Validation? White-box or Black-box?

2. A system requires a password length of 8–16 characters. List five specific test inputs for the password field using Boundary Value Analysis, labelling each as Normal, Boundary, or Erroneous.
3. A hospital patient management system is being replaced. The old system manages life-critical drug dosage data. The new system must produce identical dosage calculations for 3 months before the old system is switched off. Which conversion strategy is this? Why is Direct Changeover unacceptable here?
4. Draw a Trace Table for the following algorithm with inputs `N=3` and `N=0`.
   - Input N
   - Sum = 0
   - For I = 1 to N
   - Sum = Sum + I
   - End For
   - Output Sum
5. Explain the difference between Load Testing and Stress Testing.

SOLUTION **Exercise 1**

Process: Dry Running / Desk Checking. Type: Verification (checking logic against design). Box: White-Box (code visible).

SOLUTION **Exercise 2**

- 10 chars (Normal)
- 8 chars (Boundary Min - Valid)
- 7 chars (Boundary Min-1 - Erroneous)
- 16 chars (Boundary Max - Valid)
- 17 chars (Boundary Max+1 - Erroneous)
- Also acceptable: 0 chars / Empty string (Erroneous), 100 chars (Erroneous).

SOLUTION **Exercise 3**

Strategy: Parallel Running. Why not Direct: Direct changeover offers no fallback. If the new system calculates a dosage incorrectly, a patient could die before the error is detected. Parallel running allows comparison of outputs (Validation) ensuring safety before cutover. Cost is secondary to safety here.

SOLUTION **Exercise 4**

Input N=3:

Line	N	I	Sum
Input	3		
Init	3		0
Loop 1	3	1	1
Loop 2	3	2	3
Loop 3	3	3	6
Loop End	3	4 (Exit)	6
Output	3	4	6

Input N=0:

Line	N	I	Sum
Input	0		
Init	0		0
Loop Check	0	1 (1>0 False)	0
Output	0	1	0

Note: Boundary case N=0 executes loop zero times.

SOLUTION **Exercise 5**

Load Testing: Testing system behaviour under expected normal/peak load (e.g., 500 concurrent users). Goal: Verify performance metrics (Response time < 2s) are met under standard conditions. Stress Testing: Testing system behaviour beyond normal capacity (e.g., 2000 concurrent users) until it breaks. Goal: Identify breaking point, recovery behaviour, and data integrity under extreme pressure.

4 INFORMATION SYSTEM MANAGEMENT

Once a system has been tested and deployed in the organisation, a new phase begins — and it is by far the longest one. A student management system installed in a school in Buea, or a tax-collection system deployed in a ministry in Yaoundé, will be used for ten, fifteen, sometimes twenty years. During all that time, someone must keep it running, correct its faults, adapt it to new regulations, protect its data, train its users and check that it still serves the organisation's goals. That continuous work is the object of this final section: the **management of an information system**.

DEFINITION **Information System Management**

Information system management is the set of activities through which an organisation plans, organises, operates, secures, maintains and evaluates its information system so that the system continuously supports the organisation's strategic goals, at an acceptable cost and level of risk.

The key idea is alignment: an IS is not managed for its own sake but so that it serves the mission of the enterprise or ministry. If the Ministry of Secondary Education wants faster production of examination results, the IS department must translate that goal into concrete actions: more powerful servers, better networks, trained staff.

4.1 The IS Department and Its Actors

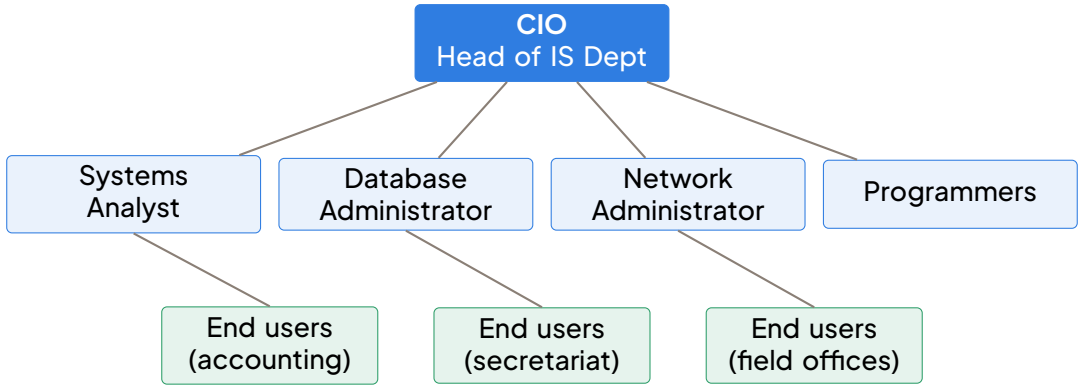
In a Cameroonian enterprise (a bank in Douala, a brewery, a mobile-phone operator) or in a ministry, the information system is looked after by the **IS department** (also called the IT department or Direction des Systèmes d'Information). Its structure depends on the size of the organisation, but the roles below are found almost everywhere.

DEFINITION **The Chief Information Officer**

The **Chief Information Officer (CIO)** is the head of the IS department. His or her mission is to define the IS strategy in line with the organisation's goals, manage the IS budget and staff, decide on investments (hardware, software, services), and guarantee the security, availability and evolution of the system.

The main roles and their responsibilities are summarised below — a classic GCE matching item.

Role	Main responsibilities
CIO	Strategy, budget, alignment with organisational goals, relations with general management.
Systems analyst	Studies user needs, analyses existing processes, writes specifications, acts as a bridge between users and developers.
Database administrator (DBA)	Installs and tunes the DBMS, manages user accounts and rights, performs backups, guarantees data integrity and availability.
Network administrator	Installs, configures and monitors the LAN/WAN, manages cabling, routers, switches, Internet access.
Programmer / developer	Writes, tests and corrects programs according to the analyst's specifications.
End user	Uses the system daily, enters data, reports faults and suggests improvements.



Simplified organisational chart of an IS department: the CIO supervises the technical roles, who in turn support the end users.

4.2 Maintaining the Information System

Maintenance is all the work done on a system after its deployment. There are four recognised types.

DEFINITION **The Four Types of Maintenance**

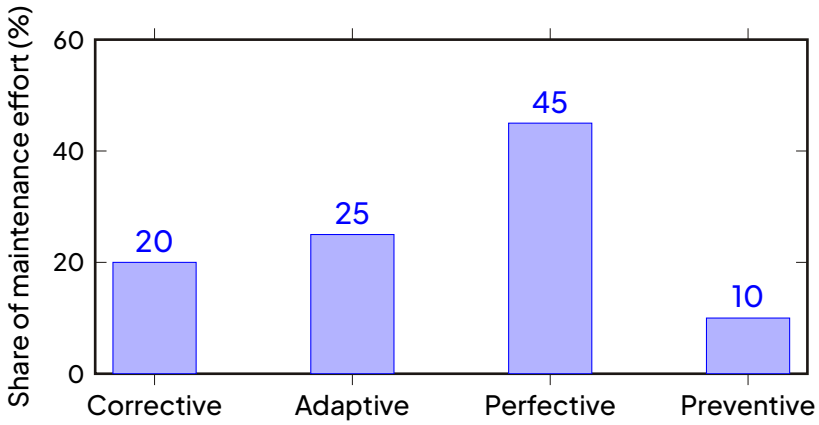
- **Corrective maintenance**: fixing faults (bugs) discovered during operation. Example: the school report-card program crashes when a class has more than 60 students; the programmer corrects the code.
- **Adaptive maintenance**: modifying the system so it continues to work when its envi-

ronment changes. Example: a payroll program is updated when the tax rates or the labour regulations change, or when the operating system is upgraded.

- **Perfective maintenance**: improving the system — new features, better performance, a nicer interface — in response to user requests. Example: adding SMS notification of results to the school system.
- **Preventive maintenance**: acting before faults occur, to avoid future problems. Example: restructuring an old, poorly documented program, replacing ageing hard disks, updating antivirus definitions.

PROPERTY **Distribution of Maintenance Effort**

Experience across the industry shows that perfective and adaptive maintenance together usually consume more resources than corrective maintenance: most maintenance money is spent evolving and adapting the system, not repairing it. Typical indicative proportions are: corrective 20%, adaptive 25%, perfective 45%, preventive 10%. (Proof not examinable; know the statement and the typical proportions.)



Typical distribution of maintenance effort: perfective and adaptive work dominate (indicative values).

4.3 Data Administration

Data is the most valuable asset of an IS: hardware can be repurchased, but ten years of student records cannot. **Data administration** covers backup, recovery, archiving and data quality.

METHOD **Designing a Backup Policy**

1. What? Identify the data to protect (databases, documents, configurations) and classify it by importance.
2. When? Fix the frequency: daily incremental backups and a weekly full backup is a common scheme for critical data.
3. Where? Store copies safely. Apply the **3-2-1 rule**: keep at least 3 copies of the data, on 2 different media, with 1 copy off-site (e.g. in another town or in the cloud), so that a fire or theft in the building cannot destroy everything.
4. Who? Assign responsibility (usually the DBA), and test the restores regularly — a backup that has never been tested cannot be trusted.

Archiving means moving data that is no longer used daily (old examination archives, closed accounts) to cheaper long-term storage, while keeping it readable and retrievable. **Data integrity** means the data is accurate and consistent (no corrupted records, referential integrity respected); **data quality** means it is complete, up to date and free of duplicates — a civil-status database in which the same person appears twice is a real operational problem.

WORKED EXAMPLE 5 **Backup Retention Calculation**

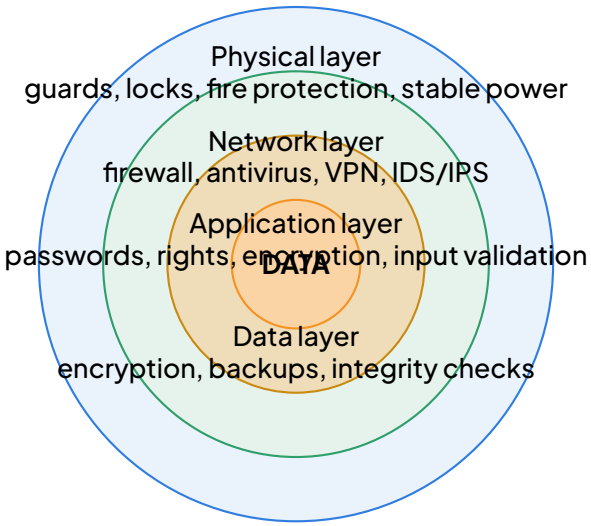
A school keeps daily incremental backups for 2 weeks and a weekly full backup for 3 months. How many backup files are stored at any time?

SOLUTION

Daily incrementals: 14 files (2 weeks × 7 days). Weekly full backups: approximately 13 files (3 months ≈ 13 weeks). Total ≈ 27 backup files. The DBA must plan storage capacity accordingly.

4.4 Security Management

Security protects the **confidentiality**, **integrity** and **availability** of the system (the **CIA triad**). It is organised in layers, like the walls of a fortress around a treasure.



Layered (defence-in-depth) security: four concentric shields around the data.

- **Physical security:** locked server rooms, security guards, fire extinguishers, surge protectors and generators against power cuts — a daily reality where electricity supply is unstable.
- **Network security:** the **firewall** filters traffic entering and leaving the organisation; **antivirus** and **anti-malware** detect and remove malicious software and must be updated constantly; **VPN** for secure remote access; **IDS/IPS** (Intrusion Detection/Prevention Systems) monitor suspicious activity.
- **Application security:** each user has an account with a strong **password** and precisely defined **user rights** (a clerk can enter marks, only the principal can validate them); sensitive systems may add **biometrics** (fingerprint, face recognition) or **two-factor authentication (2FA)**; input validation prevents injection attacks.
- **Data security:** **encryption** transforms data with a key so that it is unreadable if intercepted or stolen (encrypted connections TLS/SSL, encrypted backup disks, encrypted databases); regular **integrity checks** (checksums, hashes) detect corruption.
- **Disaster recovery plan (DRP):** a written, tested procedure describing how to restart the system after a major incident (fire, flood, cyberattack): who does what, from which backups, on which replacement machines, and within what time (**RTO** — Recovery Time Objective) and with how much data loss (**RPO** — Recovery Point Objective).

WARNING **Security Is a Process, Not a Product**

Buying an expensive firewall does not make an organisation secure. Security must be managed continuously: updates, awareness sessions, audits, tests of the disaster recovery plan. The weakest link is usually the user: a password written on paper stuck to the screen, or a clerk who opens an infected attachment, defeats the best technology.

4.5 User Support, Training and Change Management

Users need permanent support: a **help desk** (a team reachable by phone, e-mail or a ticket system) answers questions and records incidents; **documentation** (user manuals, procedures) must be updated at every system change. When a new system or version arrives, **change management** prepares people: information meetings, training sessions, pilot users, and listening to fears and objections.

> **WARNING** **The Human Factor**
>
> Neglecting user training and change management is a leading cause of IS failure — even with technically perfect software. Staff who do not understand the new system, or who fear it will take their jobs, will resist it, misuse it or quietly return to their old paper registers.

4.6 Evaluating the Information System

Management must regularly check that the IS still delivers value, using measurable **performance indicators**:

Indicator	Question it answers
Availability	What percentage of time is the system up and running? (Target often 99.5% or higher for critical systems.)
Response time	How long does a user wait for a screen or a report?
User satisfaction	Do users find the system useful and easy (surveys, help-desk tickets)?
Cost–benefit	Do the gains (time saved, fewer errors, better service) justify the running costs?

A few months after deployment, the organisation conducts a **post-implementation review**: it compares the results actually obtained with the objectives fixed at the analysis stage, identifies gaps, and decides on corrective, adaptive or perfective actions — which feeds back into the maintenance cycle.

4.7 Legal and Ethical Aspects

Managing an IS also means respecting the law and professional ethics. In Cameroon, **Law No. 2010/012 of 21 December 2010 relating to cybersecurity and cybercriminality** provides a legal framework for the security of electronic communications and the punishment of computer-related offences (illegal access to systems, data fraud, identity theft, interception of communications). Organisations that hold personal data (students' records, patients' files, customers' accounts) have an ethical and legal duty of **privacy**: collect only what is needed, protect it, and never disclose it without authorisation. Software must be used with valid **licences**: proprietary software is paid for per user or per machine, while free/open-source software may be used and modified under the conditions of its licence (e.g. GPL, MIT). Using pirated software exposes the organisation to legal sanctions and to security risks (no updates, possible malware).

WORKED EXAMPLE 6 **Matching Roles to Duties (GCE-style)**

In a ministry, the following tasks are observed: (a) defining the three-year IS investment plan; (b) performing the nightly database backup; (c) interviewing staff to write the specifications of a new application; (d) configuring the router that connects the ministry to the Internet. Match each task to the correct role: CIO, DBA, systems analyst, network administrator.

SOLUTION

- (a) CIO — strategy, planning and budget are the responsibility of the head of the IS department.
- (b) DBA — backups, user rights and data integrity belong to the database administrator.
- (c) Systems analyst — studying needs and writing specifications is the analyst's bridge role between users and developers.
- (d) Network administrator — routers, switches and connectivity are his or her domain.

Exam tip: in matching items, first eliminate the obvious ones (backup → DBA, cables/routers → network administrator), then distinguish CIO (strategy/budget) from analyst (needs/specifications).

EXERCISE 4 **Managing a School Information System**

A large secondary school in Bamenda has just deployed a system managing marks, report cards and fees.

1. Give one example of each of the four maintenance types that the school may need within three years.
2. Propose a backup policy for the marks database using the 3-2-1 rule.
3. List three performance indicators the principal could use to evaluate the system, and one security measure for each of the four layers of defence.

SOLUTION **Exercise 1**

1. Corrective: fixing a bug that swaps two students' averages. Adaptive: updating the system when the official grading scale or report-card format changes. Perfective: adding online consultation of results by parents. Preventive: replacing the ageing server disk before it fails and documenting the code.

2. What: the marks and fees database. When: incremental backup every evening, full backup every weekend. Where: 3 copies (the live database plus two backups) on 2 media (server disk and external drive), 1 copy kept off-site, e.g. at the proprietor's office in another quarter. Who: the DBA, with a restore test every term.
3. Indicators: availability (system accessible during the report-card period), response time of the marks-entry screen, teacher satisfaction survey. Security: physical — locked server room; network — firewall and updated antivirus; application — passwords and user rights (teachers enter marks, only the principal validates); data — encrypted backups.

KEY POINTS **Key Points of the Section**

- IS management keeps the system aligned with organisational goals throughout its life; the **CIO** leads the IS department (analysts, DBA, network administrator, programmers) serving the end users.
- Four maintenance types: **corrective**, **adaptive**, **perfective**, **preventive** — perfective and adaptive usually cost the most.
- Protect data with a tested backup policy (3-2-1 rule), archiving, and attention to integrity and quality.
- Security is layered (physical, network, application, data) and is a continuous process; the user is the weakest link; a disaster recovery plan must exist and be tested.
- Train users and manage change — otherwise even perfect software fails.
- Evaluate the IS with indicators (availability, response time, satisfaction, cost-benefit) and a post-implementation review; respect Cameroonian law (Law No. 2010/012) and software licences.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity closes the chapter Developing Information Systems by asking you to act, in teams, as a small software consultancy. By the end of the activity you should be able to:

- analyse a real need and write a short **requirements specification**, clearly separating **functional** from **non-functional requirements**;
- justify the choice of a **development model** (waterfall, iterative, prototyping) for a given project;
- model a system with the chapter's design tools: **context diagram**, **level-0 data flow diagram (DFD)** and **flowchart**;
- design a **test plan** covering normal, boundary and erroneous data, and choose and justify a **conversion strategy**;
- propose an **information system management plan**: administration, backups, access rights and security;
- defend your choices orally and evaluate the work of other teams against objective criteria.

KEY POINTS **Skills mobilised**

Analysis of requirements → modelling with design tools → planning of testing and implementation → organisation of system management. These are exactly the four stages of the chapter, applied to one coherent project.

5.2 Situation

Government Bilingual High School (GBHS) Bamenda is a large boarding school with about 1 200 students, of whom 800 take lunch at the school canteen every school day. Meals are paid for with **paper tickets**: a book of 20 tickets costs 2 000 FCFA (i.e. 100 FCFA per meal). Tickets are sold in cash by a cashier every morning, torn in half by the kitchen staff when a meal is served, and counted by hand every evening.

The principal, Mrs N., has identified serious problems:

- ticket books are lost, stolen, photocopied and resold; the school estimates it loses the equivalent of about 5% of sales;
- the evening manual count takes the cashier about one hour and errors are frequent;
- there is no record of which student ate on which day, so parents who pay in advance cannot be given a statement, and the bursar cannot reconcile sales with meals served;
- at peak time (12:00–12:30) a queue of up to 15 minutes forms at the cashier's desk.

Mrs N. wants to replace the paper tickets with a **computerised meal-card system**: each student receives a personal card bearing a unique number; parents credit the card at the cashier's office (cash) or by mobile money; the card is presented at the kitchen hatch, where a terminal checks the balance, deducts 100 FCFA and records the meal. The bursar must be able to print daily and monthly reports (meals served, money collected, balance per student).

Constraints stated by the principal:

- the budget is limited; the system must run on the two existing office computers and one new terminal at the hatch;
- power cuts are frequent, so no transaction may be lost when the power fails;
- the kitchen staff have basic computer skills only;
- the system must be ready for the start of the next school year, in about four months;
- students' personal data must be protected.

Your team of four has been hired to advise the principal. You will produce a mini-project dossier and defend it in a 5-minute presentation.

5.3 Tasks

Task 1 — Requirements and development model.

1. Write a one-page requirements specification containing at least **six functional requirements** (what the system must do) and at least **four non-functional requirements** (qualities: performance, usability, reliability, security, cost).
2. Which development model do you recommend: waterfall, iterative/incremental, or prototyping? Justify your choice with at least two arguments linked to this situation.

Task 2 — Modelling the solution.

3. Draw the **context diagram** of the meal-card system (the system as one process, with its external entities and the data flows between them).
4. Draw the **level-0 DFD** showing at least the processes register student, credit card, serve meal and produce reports, together with the data stores (students, accounts, meal records).
5. Draw a **flowchart** of the procedure serve a meal, from the moment a card is presented to the moment the meal is handed over or refused (include the balance check and the case of insufficient funds).

Task 3 — Testing and conversion.

6. Design a **test plan** for the serve meal function with at least **six test cases**: for each, give the input data, the expected result and the type of test (normal, boundary, erroneous). Include at least one boundary case on the balance and one erroneous case (e.g. unknown card number).
7. Choose a **conversion strategy** (direct, parallel, phased, pilot) to move from paper tickets to the card system, and justify your choice in three or four lines.

Task 4 — Management plan and presentation.

8. Propose a **management plan**: who administers the system (roles), the backup policy (what, when, where), the access rights of each user category (cashier, kitchen staff, bursar, administrator), and at least one concrete security measure.
9. Prepare a 5-minute presentation of your dossier. While other teams present, complete the critique grid below.

Criterion	Question to ask	Mark /2
Requirements	Are functional and non-functional requirements clearly distinguished and realistic?	
Models	Are the DFDs balanced and the flowchart correct (symbols, decisions)?	
Test plan	Are normal, boundary and erroneous cases all present, with expected results?	
Conversion	Is the strategy justified with respect to risk and cost?	
Management	Are roles, backups, rights and security concrete and coherent?	

5.4 Model answer

Task 1 — Requirements specification (extract).

Functional requirements:

1. The system shall register each student with a unique card number, name and class.
2. The system shall allow the cashier to credit a card by cash or mobile money and issue a receipt.
3. The system shall, when a card is presented at the hatch, check the balance, deduct 100 FCFA if sufficient, and record the meal with date and time.
4. The system shall refuse service (without deduction) if the balance is below 100 FCFA or the card is unknown or blocked.
5. The system shall produce daily and monthly reports of meals served and money collected.
6. The system shall allow the bursar to consult the balance and meal history of any student.

Non-functional requirements:

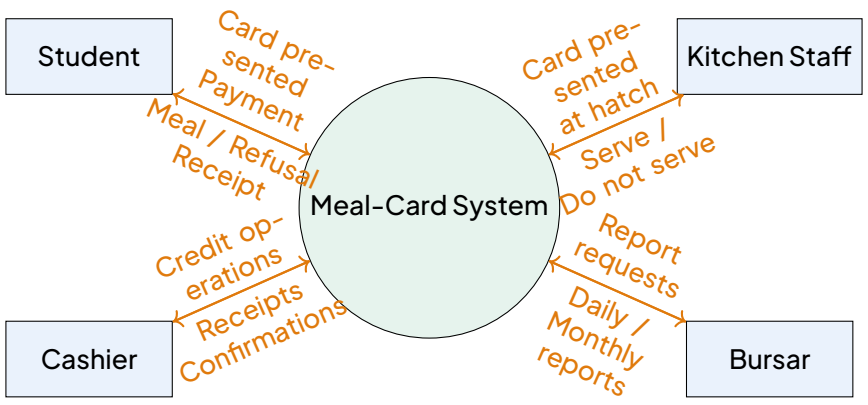
1. **Reliability:** no transaction may be lost during a power cut (transactions saved immediately, e.g. on a UPS-protected machine or with automatic recovery).
2. **Performance:** serving one meal must take less than 5 seconds so that the queue moves faster than today.
3. **Usability:** the hatch screen must be usable by staff with basic computer skills (large buttons, messages in English and French).

4. **Security:** each user logs in with a personal password; students' data are accessible only to authorised staff.

Recommended model: an **iterative/incremental model** (with a prototype of the serve meal function). Arguments: the kitchen staff have never used such a system, so an early prototype lets them react and correct misunderstandings before full development; the four-month deadline is tight, so delivering the core functions (register, credit, serve) first guarantees a usable system even if reports are finished later. A pure waterfall model would be risky because requirements would only be validated at the very end.

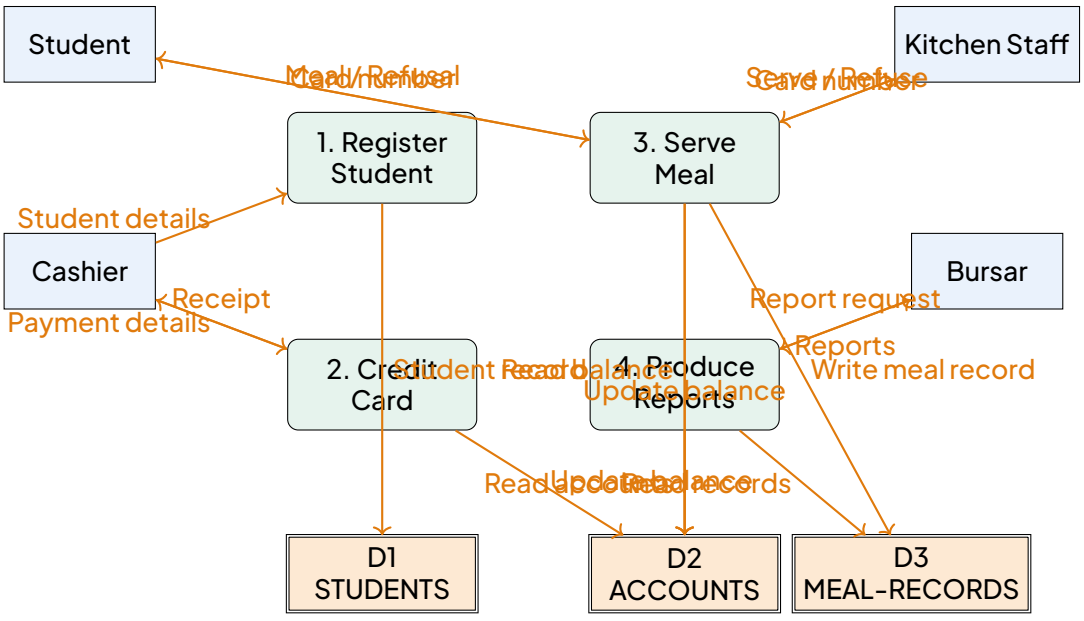
Task 2 — Models.

Context diagram. The system is represented as a single process with four external entities. The diagram below shows all data flows.



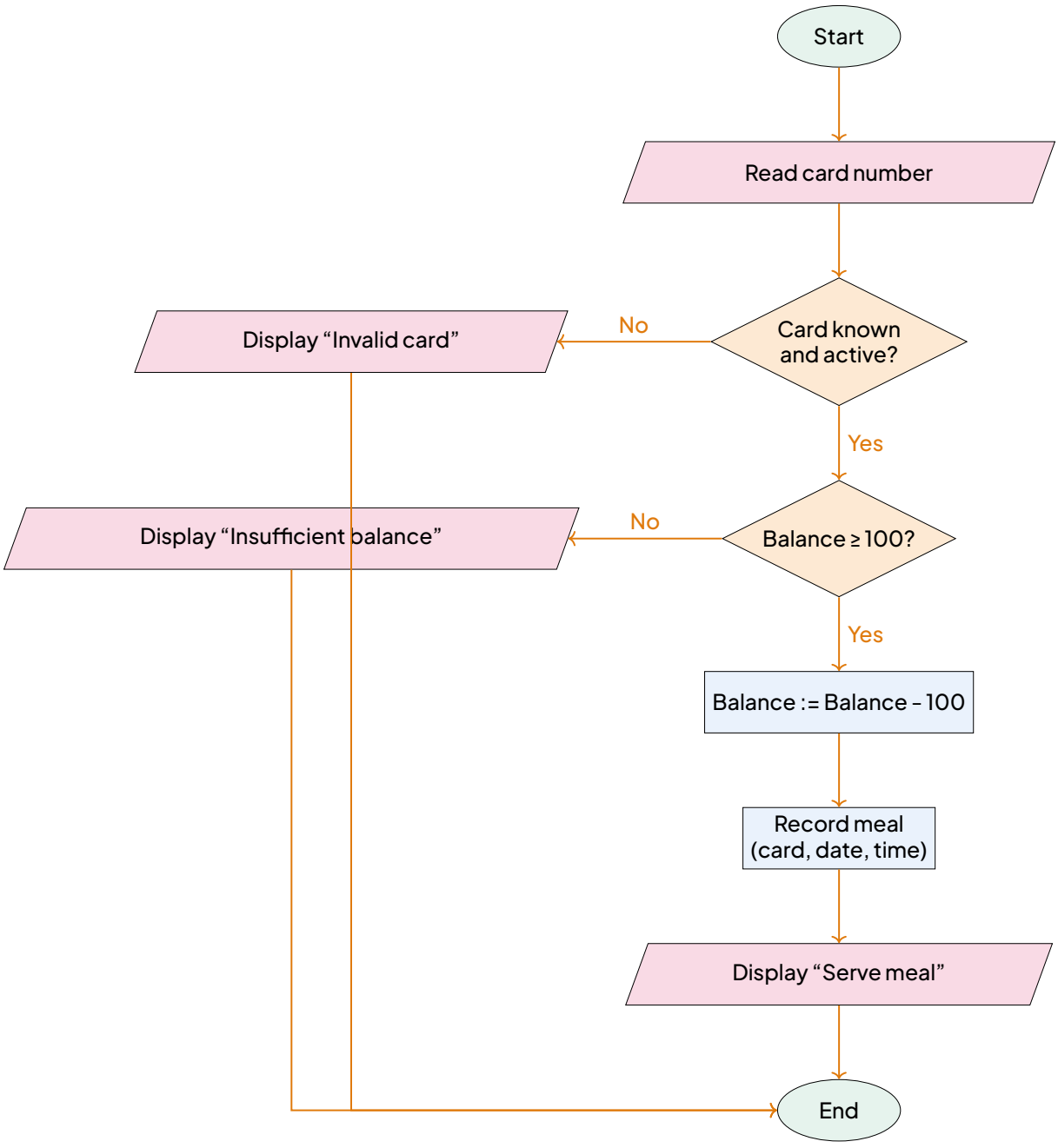
Context diagram of the Meal-Card System

Level-0 DFD. The diagram decomposes the system into four processes, three data stores and the same four external entities. It is balanced: every flow crossing the boundary of the context diagram appears here.



Level-0 DFD of the Meal-Card System

Flowchart of serve a meal. The flowchart uses standard symbols: oval (start/end), parallelogram (input/output), rectangle (process), diamond (decision).



Flowchart for the “Serve a Meal” procedure

Task 3 — Test plan and conversion.

N°	Input data	Expected result	Type
1	Valid card, balance 500	Balance becomes 400, meal recorded, “serve” displayed	Normal
2	Valid card, balance 100	Balance becomes 0, meal recorded, “serve” displayed	Boundary
3	Valid card, balance 90	Refusal, balance unchanged, no meal record, “insufficient balance” displayed	Boundary
4	Unknown card number	“Invalid card” displayed, nothing recorded	Erroneous
5	Blocked card	Refusal with “card blocked” message, no deduction	Erroneous
6	Same card presented twice within one minute	Second presentation refused or flagged as duplicate	Normal/limit
7	Power cut just after deduction	On restart, the meal record and new balance are intact (no lost transaction)	Reliability

Conversion strategy: **parallel running** for the first month of the school year. Paper tickets remain valid while the card system operates; the bursar compares the two sets of figures daily. Justification: the financial risk of direct changeover is too high (money and meals for 800 students), and a pilot on one class only would not test the peak load at the hatch; parallel running costs more effort for one month but guarantees that no student is left without a meal if the new system fails.

Task 4 — Management plan.

- **Administration:** the computer science teacher acts as system administrator (user accounts, card blocking, software updates); the cashier handles daily credit operations; the bursar supervises reports.
- **Backup policy:** automatic backup of the database every evening to an external hard disk kept in the bursar’s safe, plus a weekly copy kept off-site (e.g. at the principal’s home); a monthly restore test.
- **Access rights:** cashier — credit cards, register students; kitchen staff — serve-meal screen only; bursar — read-only reports and balances; administrator — full access. Each user has a personal login and password.
- **Security measure:** passwords plus automatic screen lock after 2 minutes of inactivity, and cards can be blocked immediately when reported lost, preventing fraudulent use.

WARNING **Frequent mistakes in this activity**

Confusing functional requirements (“deduct 100 FCFA”) with non-functional ones (“deduct it in under 5 seconds”); drawing DFD processes as people (the cashier is an external entity, not a process); forgetting the refusal branch in the flowchart; writing test

cases without the expected result; choosing direct changeover without mentioning its risk.

6 METHODS AND WORKED EXAMPLES

7 DEVELOPING INFORMATION SYSTEMS

7.1 System Development Process

DEFINITION **Information System**

An **information system** is an organised combination of people, hardware, software, communication networks, data resources, and policies and procedures that stores, retrieves, transforms, and disseminates information in an organisation.

DEFINITION **System Development Life Cycle (SDLC)**

The **System Development Life Cycle (SDLC)** is a structured, phased approach to developing an information system. It provides a framework for planning, controlling, and managing the development process from initial concept to final disposal.

PROPERTY **Stages of the SDLC**

The standard stages, in order, are:

1. Feasibility Study — determines whether the project is technically, economically, and operationally viable. Output: Feasibility Report.
2. Analysis (Investigation & Requirements Specification) — fact-finding (interviews, questionnaires, observation, document study) to understand the current system and define exactly what the new system must do. Output: Requirements Specification.
3. Design — specifies how the system will meet the requirements: system architecture, file/database structures, input/output interfaces, algorithms, and program specifications. Output: System Design Specification.
4. Implementation (Construction/Coding) — translates design into executable code; creates data files, writes programs, develops user documentation. Output: Working System + Documentation.
5. Testing — verifies that the system meets its specification (verification) and satisfies user needs (validation) using test plans, test data, and trace tables. Output: Test Re-

ports, Error Logs.

6. Deployment / Installation (Changeover) — moves the system from development to live operation using a chosen strategy (direct, parallel, phased, pilot). Output: Operational System.
7. Maintenance — modifies the system after delivery: corrective (fix bugs), adaptive (new environment), perfective (improve performance/features), preventive (avoid future faults). Output: Updated System + Revised Documentation.

Iteration: The cycle is not strictly linear; testing often reveals errors that require a return to design or implementation.

METHOD **Describing the stages of the System Development Life Cycle (SDLC)**

When a question asks you to describe, list in order or explain the stages of developing an information system, proceed as follows.

1. Recall the standard order of the SDLC: **feasibility study** → **analysis** (investigation and requirements specification) → **design** → **implementation** (coding/construction) → **testing** → **deployment/installation** → **maintenance**.
2. Give one clear sentence per stage: say what is done and what is produced (e.g. analysis produces a requirements specification; design produces system specifications, file structures and interface designs).
3. Justify the order: each stage uses the output of the previous one; you cannot design a solution before knowing the requirements.
4. Mention iteration: if testing reveals errors, the team returns to design or implementation — the cycle is not strictly linear.
5. Adapt to the scenario: name the people involved (system analyst, programmer, user) and the documents produced, using the context given in the question (school, hospital, business).

Reflex: in a "state the stage" question, look for key words: interviews/questionnaires ⇒ analysis; flowcharts, DFDs ⇒ design; coding ⇒ implementation; test data ⇒ testing; correcting errors after delivery ⇒ maintenance.

WORKED EXAMPLE 7 **Identifying SDLC stages in a scenario**

A microfinance institution in Bamenda wants a computerised loan-management system. The following activities are observed:

(a) A team studies whether the project is technically and economically possible.

(b) Staff are interviewed to find out exactly what the new system must do.

(c) Programmers write the program code.

(d) The system is run with sample loan records to check it behaves correctly.

(e) Six months after installation, a bug in the interest calculation is corrected.

For each activity, state the SDLC stage and justify your answer.

SOLUTION

Solution.

(a) Feasibility study. The team checks technical feasibility (do we have the technology and skills?), economic feasibility (do benefits exceed costs?) and operational feasibility (will staff accept and use it?). This always comes first, before any detailed work.

(b) Analysis (requirements gathering). Interviews, questionnaires, observation and document study are fact-finding techniques used to establish the requirements specification: what the system must do (record loans, compute interest, produce statements).

(c) Implementation (construction/coding). Writing program code transforms the design (algorithms, file structures) into a working system. This stage only makes sense after design is complete.

(d) Testing. Running the system with test data (sample loan records) and comparing actual results with expected results detects errors before real data is used.

(e) Maintenance. Correcting an error discovered after delivery is corrective maintenance. (Adapting the system to a new tax law would be adaptive maintenance; improving response time would be perfective maintenance.)

Examiner's remark: always give the stage name and a justification linked to the scenario; a bare list earns only half the marks.

KEY POINTS **Key Roles in SDLC**

- System Analyst: investigates current system, writes requirements specification, designs new system, liaises with users and programmers.
- Programmer: writes, tests, and debugs code according to design specifications.
- Database Administrator (DBA): designs database schema, manages security, backup, recovery, performance tuning.
- End User: provides requirements, participates in acceptance testing, uses the final

system.

- Project Manager: plans, schedules, monitors progress, manages resources and risks.

7.2 System Design Tools

DEFINITION **Algorithm**

An **algorithm** is a finite sequence of well-defined, unambiguous instructions that solves a problem or performs a computation.

METHOD **Drawing a program flowchart**

1. Identify the process: inputs, processing steps (in order), decisions, outputs.
2. Use the correct symbols:
   - Oval (terminal): Start / Stop
   - Parallelogram: Input / Output
   - Rectangle: Process (assignment, calculation)
   - Diamond: Decision (exactly two exits labelled Yes/No or True/False)
   - Arrows: Flow of control
3. Place Start at top, Stop at bottom; flow runs top to bottom.
4. For a loop: initialise before the decision diamond; the "No/continue" branch returns above the diamond; the "Yes/finished" branch exits towards Stop. Ensure some instruction inside the loop modifies the tested variable (otherwise infinite loop).
5. Check: every diamond has two labelled exits; every path ends at Stop; no crossing arrows if avoidable.

Reflex: read the problem and underline verbs: "read/enter" ⇒ parallelogram; "compute/calculate" ⇒ rectangle; "if/while/until" ⇒ diamond.

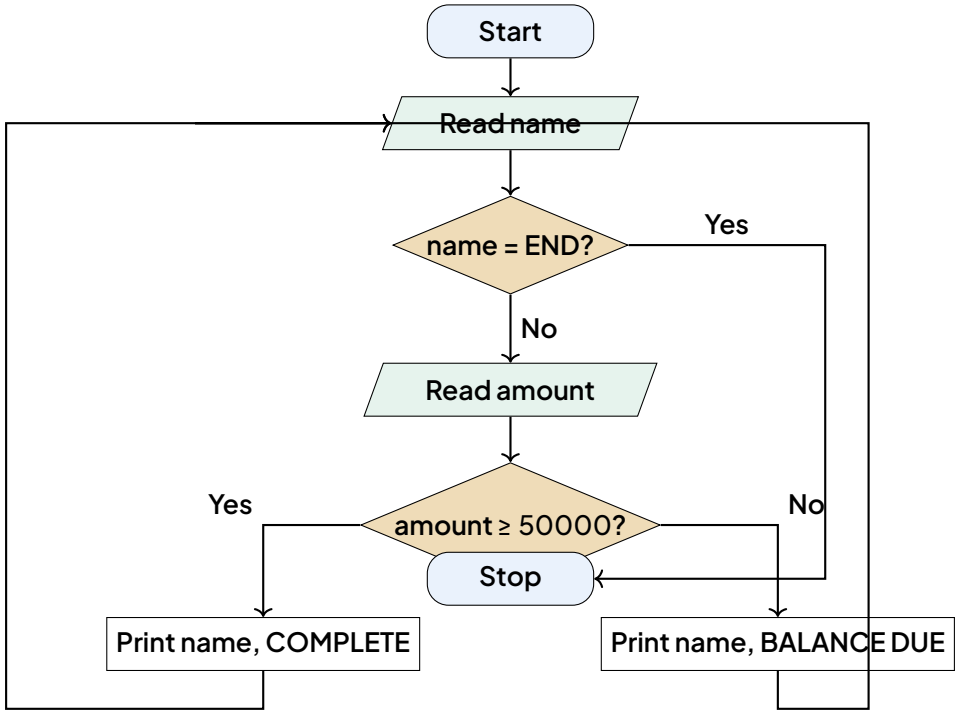
WORKED EXAMPLE 8 **Flowchart for a school fee check**

A bursar's program repeatedly reads a student's name and the amount paid, until the name entered is `END`. For each student it prints the name and "COMPLETE" if the amount

paid is at least 50 000 FCFA, otherwise it prints the name and "BALANCE DUE". Draw the flowchart.

SOLUTION

Solution. The algorithm is: start; read name; while name ≠ `END`: read amount, test amount ≥ 50000, print appropriate message, read next name; stop.



Flowchart for the school-fee checking program.

Checks performed: the sentinel value `END` is tested before reading the amount (so no amount is requested for `END`); both branches of the amount decision converge and loop back to "Read name"; the "Yes" exit of the first diamond leads to Stop. Each diamond has exactly two labelled exits.

DEFINITION **Data Flow Diagram (DFD)**

A **Data Flow Diagram (DFD)** is a graphical representation of the flow of data through an information system, modelling its process aspects. It shows what data enters and leaves the system, where it comes from and goes to, and where it is stored.

PROPERTY **DFD Symbols and Rules**

Symbol	Name	Meaning
Rectangle	External Entity	Source or destination of data outside the system (person, organisation)
Circle / Rounded Rectangle	Process	Transforms input data into output data; numbered (1, 2, 3…)
Open-ended Rectangle	Data Store	Repository of data at rest (file, database); labelled D1, D2…
Arrow	Data Flow	Movement of data; labelled with data name (never a physical object)

Rules:

1. A process must have at least one input flow and one output flow.
2. Data cannot flow directly between two external entities.
3. Data cannot flow directly between two data stores.
4. A data store is read from or written to only by a process.
5. Every data flow must be labelled with the name of the data (e.g. “mark sheet”, “report card”).

METHOD **Drawing a Data Flow Diagram (DFD)**

1. List the external entities (people or organisations that send/receive data: Student, Teacher, Bank). Draw them as rectangles.
2. List the processes (actions transforming data: Register student, Compute average). Draw them as circles or rounded rectangles, numbered (1, 2, 3…).
3. List the data stores (files/databases: Student file, Marks file). Draw them as open-ended rectangles, labelled D1, D2, …
4. Connect with labelled arrows: every arrow carries a data name (e.g. “mark sheet”, “report card”), never a physical object.
5. Verify the rules: a process must have at least one input and one output flow; data can-

not flow directly between two external entities or between two data stores without a process; a data store is read or written by a process.

Reflex: a context diagram (level 0) has one single process representing the whole system; level 1 decomposes it.

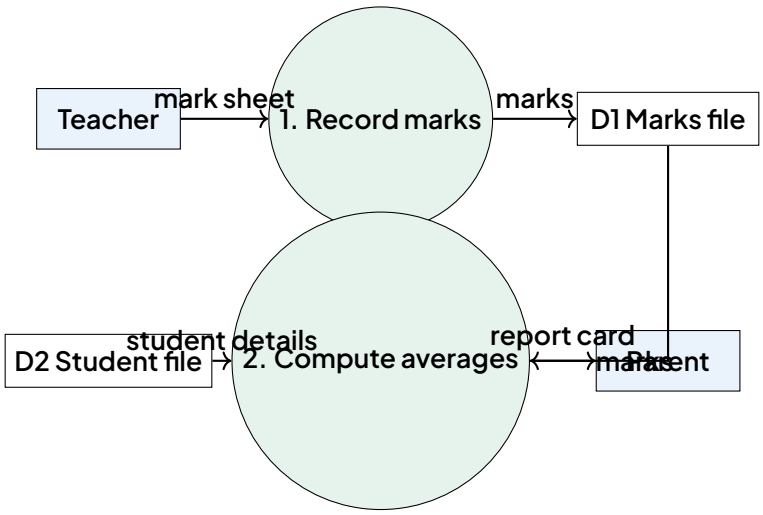
WORKED EXAMPLE 9 **DFD for a school report-card system**

In a college in Yaoundé, teachers submit mark sheets; the system records them in a Marks file, then computes averages using student information from the Student file, and finally produces report cards given to parents. Draw the level-1 DFD.

SOLUTION

Solution.

- External entities: Teacher (sends mark sheets), Parent (receives report cards).
- Processes: 1. Record marks; 2. Compute averages and produce reports.
- Data stores: D1 Marks file; D2 Student file.



Level-1 DFD of the report-card system.

Verification against the rules: process 1 has an input (mark sheet) and an output (marks to D1) — valid; process 2 reads from two data stores and outputs to an entity — valid; there is no direct arrow Teacher→Parent or D1→D2 — valid; every arrow is labelled with data. A common error is to draw an arrow from Teacher straight to the Marks file: an external entity may never write directly into a data store.

DEFINITION **Data Dictionary**

A **data dictionary** is a structured repository of definitions of all data elements used in an information system. It defines the structure, type, size, validation rules, and relationships of each data item, enabling programmers to create file/database structures without ambiguity.

METHOD **Writing a data dictionary entry**

1. For each data flow or store, list its fields.
2. For each field give: name, data type (text/string, integer, real, date, Boolean), size/length, allowed values or format (validation rule), and an example.
3. Use standard notation for composite structures:
   - = "is composed of"
   - + "and" (sequence)
   - { } "repetition" (one or more occurrences)
   - [|] "choice" (one of the alternatives)
   - ( ) "optional" (may be absent)
4. Identify the primary key of each file (unique identifier).

Reflex: a good dictionary entry must let a programmer create the file structure without asking any question.

WORKED EXAMPLE 10 **Data dictionary for a Student file**

The Student file of a school information system stores for each student: a unique 8-character registration number, full name (up to 40 characters), date of birth, sex (M or F), class, and fees paid (an amount in FCFA). Write the data dictionary entry for this file.

SOLUTION

Solution.

Student record = RegNo + FullName + DateOfBirth + Sex + Class + FeesPaid

Field	Type	Size	Validation rule	Example
RegNo	Text	8	Unique; not empty (primary key)	GS2024A1
FullName	Text	40	Not empty	Ngo Bessala Marie
DateOfBirth	Date	8	Format DD/MM/YYYY; not in future	12/03/2007
Sex	Text	1	M or F only	F
Class	Text	6	Must exist in Class file	USc A
FeesPaid	Integer	7	0 ≤ FeesPaid ≤ 9 999 999	50000

Comments: RegNo is chosen as **primary key** because it is unique and never changes (a name is not unique). Sex uses a choice notation: Sex = [M | F]. The validation rules given here (presence check, format check, range check, existence check) are exactly the checks the system will apply at data entry — this links the data dictionary to data validation, a favourite GCE topic.

WORKED EXAMPLE 11 **Data dictionary with repetition and choice**

A library system stores a `Book` record composed of: ISBN (13 digits, primary key), Title (up to 80 chars), Authors (one or more names, each up to 40 chars), Category (Fiction, Non-Fiction, Reference), and an optional PublicationYear (4 digits). Write the data dictionary entry using standard notation.

SOLUTION

Solution.

Book = ISBN + Title + {Author} + Category + (PublicationYear)

Author = AuthorName (Text, 40 chars)

Category = [Fiction | Non-Fiction | Reference]

Field	Type	Size	Validation rule	Example
ISBN	Text	13	13 digits; unique (PK)	9782070368228
Title	Text	80	Not empty	L'Étranger
AuthorName	Text	40	Not empty	Albert Camus
Category	Text	14	Fiction | Non-Fiction | Reference	Fiction
PublicationYear	Integer	4	Optional; 1000–2025	1942

7.3 System Testing and Implementation

DEFINITION **Verification and Validation**

- Verification: "Are we building the system right?" — checking that the system meets its specification (reviews, inspections, walkthroughs, testing against specs).
- Validation: "Are we building the right system?" — checking that the system meets the user's actual needs (acceptance testing with real data).

METHOD **Designing a test plan and choosing test data**

1. State the objective of the test (what is being verified).
2. Choose three categories of test data:
   - Normal (valid) data: typical values the system must accept.
   - Extreme (boundary) data: values at the limits of validity (e.g. mark = 0 and mark = 20).
   - Erroneous (invalid) data: values that must be rejected (e.g. mark = 25, mark = "ABC").
3. For each test, record in a table: test number, purpose, test data, expected result, actual result, pass/fail.
4. Compare expected and actual results; any mismatch means a bug to correct (return to implementation).
5. Distinguish the levels:
   - **Unit testing**: each module alone (white-box, by programmer).
   - **Integration testing**: modules combined (interfaces, data passing).
   - **System testing**: whole system against requirements specification.
   - **Acceptance testing**: by the user, with real data — alpha testing (in-house) and beta testing (selected outside users).

Reflex: expected results must be computed by hand before running the test, never copied from the program's output.

WORKED EXAMPLE 12 **Test plan for a mark-validation module**

A module accepts a GCE mock mark out of 20 and displays "VALID" or "INVALID". Design a test plan with at least five tests covering all categories of test data.

SOLUTION

Solution. Valid marks are real numbers with $0 \le m \le 20$.

No	Purpose	Data	Expected result	Pass?
1	Normal value accepted	12.5	VALID	Yes
2	Lower boundary	0	VALID	Yes
3	Upper boundary	20	VALID	Yes
4	Just above upper limit	20.5	INVALID	Yes
5	Negative value rejected	-3	INVALID	Yes
6	Non-numeric input rejected	ABC	INVALID (error message)	No

Analysis: tests 1–3 check valid and boundary data; tests 4–6 check erroneous data. Test 6 failed: the program crashed instead of displaying "INVALID". Conclusion: the module needs a type check before the range check; the programmer returns to the implementation stage, corrects the code, and the whole test plan is re-run (regression testing), because a correction can break what previously worked.

METHOD **Dry-running an algorithm with a trace table**

1. Draw one column per variable (including loop counters and outputs), one row per executed instruction that changes a value.
2. Execute the algorithm mentally, line by line, writing each new value in the table; cross out or leave blank unchanged values.
3. At each decision, evaluate the condition with the current values and follow the correct branch.
4. Count loop iterations carefully; check the first and the last iteration (most errors hide there).
5. Conclude: final values, printed output, and whether the algorithm is correct (e.g. off-by-one errors, variables not initialised).

WORKED EXAMPLE 13 **Trace table: counting passed students**

The following algorithm processes the marks of 5 students: 12, 7, 15, 9, 10. A student passes with mark ≥ 10.

```
count ← 0
FOR i ← 1 TO 5 DO
  read mark
  IF mark ≥ 10 THEN count ← count + 1
ENDFOR
print count
```

Dry-run the algorithm and give the output.

SOLUTION

Solution. Trace table:

i	mark	mark ≥ 10?	count
–	–	–	0
1	12	Yes	1
2	7	No	1
3	15	Yes	2
4	9	No	2
5	10	Yes	3

After the loop, `print count` displays 3. Note the last iteration: mark = 10 satisfies ≥ 10, so it is counted — a frequent error is to treat the boundary as excluded. The algorithm is correct: exactly the students with 12, 15 and 10 pass.

WORKED EXAMPLE 14 **Trace table with nested loop: finding maximum**

Dry-run the following algorithm with the input list: 4, 9, 2, 9, 5.

```
max ← -1
FOR i ← 1 TO 5 DO
  read x
  IF x > max THEN max ← x
ENDFOR
```

```
print max
```

SOLUTION

Solution.

i	x	x > max?	max
–	–	–	-1
1	4	Yes	4
2	9	Yes	9
3	2	No	9
4	9	No	9
5	5	No	9

Output: 9. Note: when `x = 9` the second time, the condition `x > max` is false (9 > 9 is false), so `max` stays 9 — the algorithm correctly keeps the first occurrence of the maximum.

METHOD **Choosing an implementation (changeover) strategy**

1. Know the four strategies:
   - Direct changeover: old system stopped, new one started immediately ("big bang").
   - Parallel running: old and new run together for a period; results compared.
   - Phased changeover: the new system is introduced module by module (e.g. payroll first, then inventory).
   - Pilot changeover: the new system is tried in one branch/site/department first.
2. Weigh risk against cost: direct is cheapest but riskiest; parallel is safest but most expensive (double work).
3. Match to the scenario: critical systems (hospital, bank, exam results) ⇒ parallel or pilot; small low-risk systems ⇒ direct; very large systems ⇒ phased.
4. Justify your choice in the exam with at least one advantage linked to the scenario and one drawback accepted.

WORKED EXAMPLE 15 **Choosing a changeover strategy**

A regional hospital in Buea computerises its patient-billing system, which currently runs

manually. Billing errors could endanger finances and patient trust. The hospital has limited funds and only one site. Recommend a changeover strategy, with two reasons, and state one disadvantage the hospital must accept.

SOLUTION

Solution. Recommended strategy: parallel running. For a trial period (e.g. one month), the manual billing continues and the new system runs alongside; the bills produced by both are compared daily.
Reason 1 (safety): billing is critical — if the new system fails or computes wrongly, the manual system still produces correct bills, so no patient is affected and no revenue is lost. Direct changeover would be too risky here.
Reason 2 (validation with real data): comparing outputs on real transactions is the most convincing acceptance test; discrepancies reveal bugs that test data missed.
Disadvantage accepted: parallel running is expensive and tiring — staff must do every task twice (double data entry), which increases workload and cost during the changeover period. Since the hospital has a single site, pilot changeover is impossible, and phased changeover is less suitable because billing is one indivisible function.

7.4 Information System Management

DEFINITION **Information System Management**

Information System Management encompasses the activities, policies, and procedures required to operate, secure, maintain, and evolve an information system throughout its operational life.

PROPERTY **Four Pillars of IS Management**

1. People Management: roles (system administrator, database administrator, network administrator, help desk, end users), training, access rights, separation of duties.
2. Data Management: data integrity (accuracy, consistency), backup and recovery procedures, archiving, data quality audits.
3. Security Management: physical security, logical security (authentication, authorisation, encryption, audit trails), malware protection, incident response.
4. Maintenance Management: corrective, adaptive, perfective, preventive maintenance; version control; change management procedures.

METHOD **Answering questions on Information System Management**

1. Structure the answer around the four management concerns: people (training, roles: system administrator, database administrator, users), data (backup, recovery, data integrity), security (physical security, passwords, access rights, encryption, antivirus, audit trails) and maintenance (corrective, adaptive, perfective, preventive).
2. Define each term in one line before applying it.
3. Distinguish the maintenance types carefully — examiners test this distinction:
   - **Corrective maintenance** = fix bugs/errors discovered after delivery.
   - **Adaptive maintenance** = adjust to a changed environment (new law, new OS, new hardware).
   - **Perfective maintenance** = improve performance, add features, enhance usability.
   - **Preventive maintenance** = act before faults occur (refactoring, applying patches, monitoring).
4. For backup questions, state: what is backed up, how often (daily incremental, weekly full), where stored (off-site, fireproof safe, cloud), and who is responsible.
5. Always link to the scenario: "the DBA grants read-only access to teachers but read-write access to the bursar" is worth more than a generic sentence.

WORKED EXAMPLE 16 **Managing a school's information system**

A secondary school in Douala has installed a student information system holding marks, fees and medical data. (a) Propose three security measures, each justified. (b) The Ministry introduces a new grading scale; the system must be modified. Which type of maintenance is this? Justify. (c) Design a simple backup policy.

SOLUTION

Solution. (a) Security measures:

1. User authentication with role-based access rights: each staff member has a username and password; teachers can enter marks but cannot see medical records; only the bursar modifies fees. Justification: confidentiality of sensitive data (medical data) and data integrity (only authorised persons modify).
2. Antivirus software and firewall, kept up to date: protects the server from malware introduced via USB keys or the Internet.
3. Physical security: the server room is locked, with a surge protector/UPS against power cuts (frequent in Douala). Justification: prevents theft and data loss from sud-

den power failure.

(b) Type of maintenance: adaptive maintenance. The system itself has no bug; it must be changed because the environment changed (a new official grading scale). Corrective maintenance would apply only if the old scale were wrongly coded. (c) Backup policy:

- A full backup of the database every Friday evening onto an external drive.
- An incremental backup every evening (only data changed that day).
- One copy kept off-site (e.g. at the proprietor's residence or in cloud storage) so that a fire or theft at school cannot destroy both copies.
- The system administrator is responsible, and a restore test is performed once per term to verify the backups are usable — a backup that cannot be restored is worthless.

Examiner's remark: notice how each measure is justified and anchored in the scenario; this is what converts a mediocre answer into full marks.

WORKED EXAMPLE 17 **Maintenance type identification**

Classify each situation:

1. A program crashes when a date of 29/02 is entered in a non-leap year.
2. The government changes the VAT rate from 19.25% to 20%.
3. Users request a "dark mode" interface for reduced eye strain.
4. The IT team refactors a module to remove duplicated code before it causes bugs.

SOLUTION

Solution.

1. Corrective — a bug (missing leap-year validation) is fixed.
2. Adaptive — the legal/regulatory environment changed.
3. Perfective — enhancement requested by users, not a bug fix.
4. Preventive — proactive improvement to avoid future faults.

WARNING **Frequent errors in this chapter**

- Confusing analysis (what the system must do) with design (how it will do it).
- Drawing DFD arrows between two data stores or two external entities — always pass

through a process.

- Testing only with valid data: boundary and erroneous data are where bugs hide.
- Confusing verification (are we building the system right? — reviews, tests against specifications) with validation (are we building the right system? — user acceptance).
- Calling every modification "maintenance" without specifying the type — always say corrective, adaptive, perfective or preventive.
- In trace tables, forgetting that a loop counter changes before the body is re-executed.
- Forgetting to label both exits of a decision diamond in a flowchart.
- Omitting the primary key in a data dictionary entry.
- Not justifying the choice of changeover strategy with scenario-specific reasons.

DID YOU KNOW? **Cameroon Context: GCE Practical Projects**

In the GCE Advanced Level Computer Science practical project (Paper 3), you are expected to demonstrate the full SDLC: write a feasibility study, produce a requirements specification, draw DFDs and flowcharts, create a data dictionary, implement the system in a high-level language (Python, VB.NET, C++), design and execute a test plan (including trace tables), and write a user manual. The project report is marked against these exact stages — mastering this chapter is therefore essential for both the written paper (Paper 2) and the project.

KEY POINTS **Chapter Summary: CSC14 Developing Information Systems**

- SDLC stages: Feasibility → Analysis → Design → Implementation → Testing → Deployment → Maintenance (iterative).
- Design tools: Flowcharts (control flow), DFDs (data flow), Data Dictionary (data definitions).
- DFD rules: Process = transform; Entity = source/sink; Store = data at rest; Flows = labelled data; No direct Entity↔Entity or Store↔Store.
- Testing: Normal, Extreme, Erroneous data; Unit → Integration → System → Acceptance; Trace tables for dry-runs.
- Changeover: Direct, Parallel, Phased, Pilot — choose based on risk, cost, criticality.
- Management: People, Data, Security, Maintenance (Corrective, Adaptive, Perfec-

tive, Preventive).

- Backup: Full + Incremental; Off-site copy; Regular restore tests; Clear responsibility.

8 EXERCISES

8.1 I check my knowledge

EXERCISE 5 **Multiple choice questions**

For each question, choose the single correct answer.

1. In the System Development Life Cycle (SDLC), the stage in which the analyst studies the existing system and collects the requirements of the users is called:

   (a) implementation

   (b) analysis

   (c) maintenance

   (d) testing

2. In a Data Flow Diagram (DFD), a rectangle with sharp corners (external entity) represents:

   (a) a data store

   (b) a process

   (c) a person or organisation outside the system

   (d) a data flow

3. Testing performed by real users in their own environment, before the final release of the software, is called:

   (a) alpha testing

   (b) unit testing

   (c) beta testing

   (d) white-box testing

4. Maintenance carried out to correct errors discovered after the software has been installed is called:

   (a) adaptive maintenance

   (b) corrective maintenance

   (c) perfective maintenance

   (d) preventive maintenance

EXERCISE 6 **True or false**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. In a flowchart, a parallelogram is used to represent a decision.
2. Parallel running means that the old system and the new system operate together for some time.
3. The feasibility study is carried out after the new system has been fully implemented.
4. A Database Administrator (DBA) is responsible for the security, backup and performance of the database.

EXERCISE 7 **Definitions**

Define each of the following terms in one or two clear sentences:

1. information system;
2. System Development Life Cycle (SDLC);
3. system testing;
4. verification and validation (state the difference between them);
5. the waterfall model.

EXERCISE 8 **Direct application: marks validation**

A program must read a mark m (a real number) and reject it if it is not in the range 0 to 100.

1. State the set of valid values for m.
2. Give one boundary value that must be tested at the lower limit and one at the upper limit.
3. Give two erroneous values that the program must reject.
4. Write, in pseudocode, the decision structure that displays `"Invalid mark"` when m is out of range.

8.2 I practise

EXERCISE 9 **SDLC for a hospital**

The Buea Regional Hospital wants to computerise its patient records. List the six stages of the SDLC in their correct order and, for each stage, state one activity that will be carried out in this hospital project and one deliverable (document or product) it produces.

EXERCISE 10 **Context diagram and level-0 DFD**

The GCE Board wants an online registration system: candidates register through their schools, schools submit registration files, the GCE Board validates them, and payment records are kept.

1. Identify the external entities and the main data stores of the system.
2. Draw the context diagram of the system.
3. Draw the level-0 DFD showing at least three processes (registration, validation, payment recording).

EXERCISE 11 **Completing a flowchart**

A flowchart must read a student's mark (0–100) and display the grade: `"A"` if mark ≥ 80, `"B"` if 60 ≤ mark < 80, `"C"` if 40 ≤ mark < 60, and `"F"` otherwise.

1. Draw the complete flowchart using the correct symbols (terminator, input/output, process, decision, arrows).
2. A classmate drew a flowchart in which he used a parallelogram for the decisions, a diamond for the input of the mark, and forgot the `Start` symbol. Identify and correct each symbol error.

EXERCISE 12 **Designing test cases**

A module computes the average of three subject marks (each out of 20) and decides `"Pass"` if the average is at least 10 out of 20, otherwise `"Fail"`. Design six test cases presented in a table with columns: test number, input data (the three marks), expected output, and type of test (normal, boundary or erroneous).

8.3 I prepare for the GCE

EXERCISE 13 Structured question: computerising a college

A Government High School in Bamenda wants to replace its paper-based result processing with a new information system.

1. Define the term information system. (2 marks)
2. Name the stages of the SDLC and state one activity performed at each stage in this project. (6 marks)
3. The school hesitates between direct changeover, parallel running, phased conversion and pilot conversion. Explain each method. (8 marks)
4. Recommend the most suitable method for this school and give two justifications for your choice. (4 marks)

EXERCISE 14 Testing and IS personnel

A software company in Douala has just developed a mobile banking application.

1. Distinguish between verification and validation, illustrating each with this application. (4 marks)
2. Distinguish between alpha testing and beta testing, stating who performs each and where. (4 marks)
3. Match each job title to its main duty: Chief Information Officer (CIO), Database Administrator (DBA), systems analyst, network administrator, help-desk technician. Duties: (i) aligns IT strategy with the goals of the organisation; (ii) studies user needs and specifies new systems; (iii) installs and maintains the network equipment; (iv) manages, secures and backs up the databases; (v) assists users with everyday technical problems. (5 marks)
4. Classify each of the following maintenance scenarios as corrective, adaptive, perfective or preventive: (i) fixing a bug that doubles some transactions; (ii) adapting the application to a new version of Android; (iii) adding a fingerprint login requested by users; (iv) reorganising the database indexes before performance degrades; (v) correcting a wrong currency-conversion formula. (5 marks)

EXERCISE 15 Essay question

“A good information system is 20% technology and 80% management.”
Discuss this statement with reference to security, staff training, documentation and user

involvement in the development and management of information systems. Your essay should have an introduction, at least four developed arguments illustrated with concrete examples (you may refer to Cameroonian schools, banks or hospitals), and a conclusion giving your own justified position. (20 marks)

9 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Answer: (b) analysis. The analysis stage involves studying the existing system, gathering user requirements, and defining what the new system must do.
2. Answer: (c) a person or organisation outside the system. In a DFD, an external entity (rectangle with sharp corners) represents a source or destination of data outside the system boundary.
3. Answer: (c) beta testing. Beta testing is performed by real users in their own environment before final release; alpha testing is done internally by developers.
4. Answer: (b) corrective maintenance. Corrective maintenance fixes errors discovered after installation; adaptive maintenance adapts to environment changes; perfective maintenance improves performance or adds features; preventive maintenance aims to prevent future problems.

SOLUTION **Exercise 2 — True or false**

1. False. In a flowchart, a diamond (rhombus) represents a decision; a parallelogram represents input/output.
2. True. Parallel running (parallel changeover) operates the old and new systems simultaneously for a period to compare results and ensure reliability.
3. False. The feasibility study is carried out before development begins, during the preliminary investigation/analysis phase, to decide whether the project is viable.
4. True. A Database Administrator (DBA) is responsible for database security, backup/recovery, performance tuning, and integrity.

SOLUTION **Exercise 3 — Definitions**

1. Information system: An organised combination of people, hardware, software, communication networks, data resources, and policies that collects, processes, stores, and disseminates information to support decision-making and control in an organisation.

2. System Development Life Cycle (SDLC): A structured, phased approach to developing an information system, typically consisting of stages such as preliminary investigation, feasibility study, analysis, design, implementation, and maintenance.
3. System testing: The process of evaluating the complete, integrated system to verify that it meets specified requirements, usually performed after unit and integration testing.
4. Verification and validation:
   - Verification: Checking whether the system is built right (i.e., conforms to its specification). Example: reviewing code against design documents.
   - Validation: Checking whether the right system is built (i.e., meets the user's actual needs). Example: user acceptance testing with real data.
5. Waterfall model: A linear, sequential SDLC model where each phase (requirements, design, implementation, testing, deployment, maintenance) must be completed before the next begins, with little or no overlap or iteration. Examinable: description and limitations.

SOLUTION **Exercise 4 — Direct application: marks validation**

1. The set of valid values for m is the closed interval $[0, 100]$, i.e., $0 \leq m \leq 100$.
2. Boundary values: lower limit 0, upper limit 100. (These are the exact boundaries; boundary-value testing uses 0 and 100 as the boundary values themselves, and often also -1 and 101 as the immediate outside values.)
3. Erroneous values (outside the range): e.g., -5 and 105 (or any $m < 0$ or $m > 100$).
4. Pseudocode (guard-clause style):

```
INPUT m
IF m < 0 OR m > 100 THEN
  DISPLAY "Invalid mark"
END IF
// continue processing valid mark
```

(Equivalently: `IF NOT (m >= 0 AND m <= 100) THEN DISPLAY "Invalid mark" END IF`)

SOLUTION **Exercise 5 — SDLC for a hospital**

The six stages of the SDLC in order, with one activity and one deliverable for the Buea Regional Hospital project:

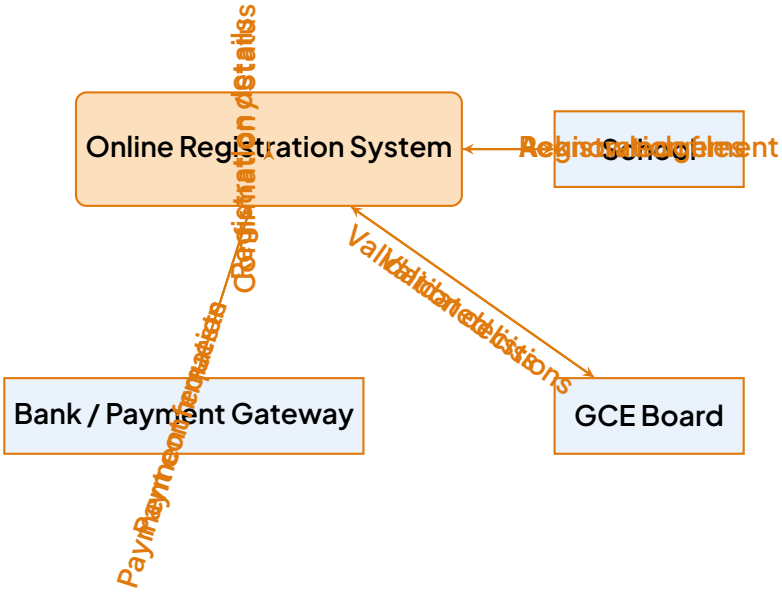
Stage	Activity in this project	Deliverable
1. Preliminary Investigation / Planning	Identify the need to computerise patient records; define project scope and objectives.	Project charter / Terms of reference
2. Feasibility Study	Assess technical, economic, operational, legal, and schedule feasibility of the new system.	Feasibility report with recommendation
3. Analysis	Interview doctors, nurses, administrators; document current manual processes and user requirements.	Requirements specification document (SRS)
4. Design	Design database schema for patients, visits, prescriptions; design input screens, output reports, and security controls.	System design document (technical specs, ER diagrams, DFDs, UI mockups)
5. Implementation	Code the software, create the database, migrate existing paper records, train staff, install hardware.	Working system, user manuals, training materials
6. Maintenance	Fix bugs reported by users, apply security patches, add new features (e.g., SMS reminders), optimise queries.	Maintenance logs, updated documentation, new releases

SOLUTION **Exercise 6 — Context diagram and level-0 DFD**

1. External entities:
   - Candidate (or Student)
   - School (registration officer)
   - GCE Board (validation officer)
   - Bank / Payment gateway (for payment processing)

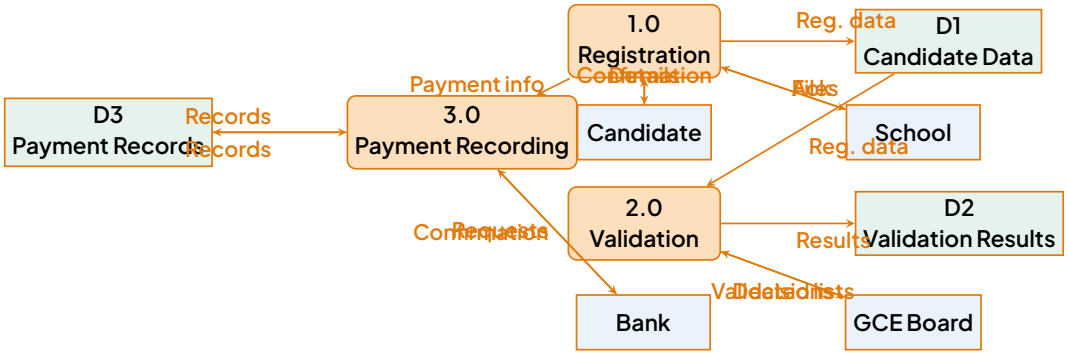
   Main data stores:
   - Candidate/Registration data store
   - School data store
   - Payment records data store
   - Validation results data store

2. Context diagram (Level-0 DFD with a single process):



Context diagram for the GCE Online Registration System

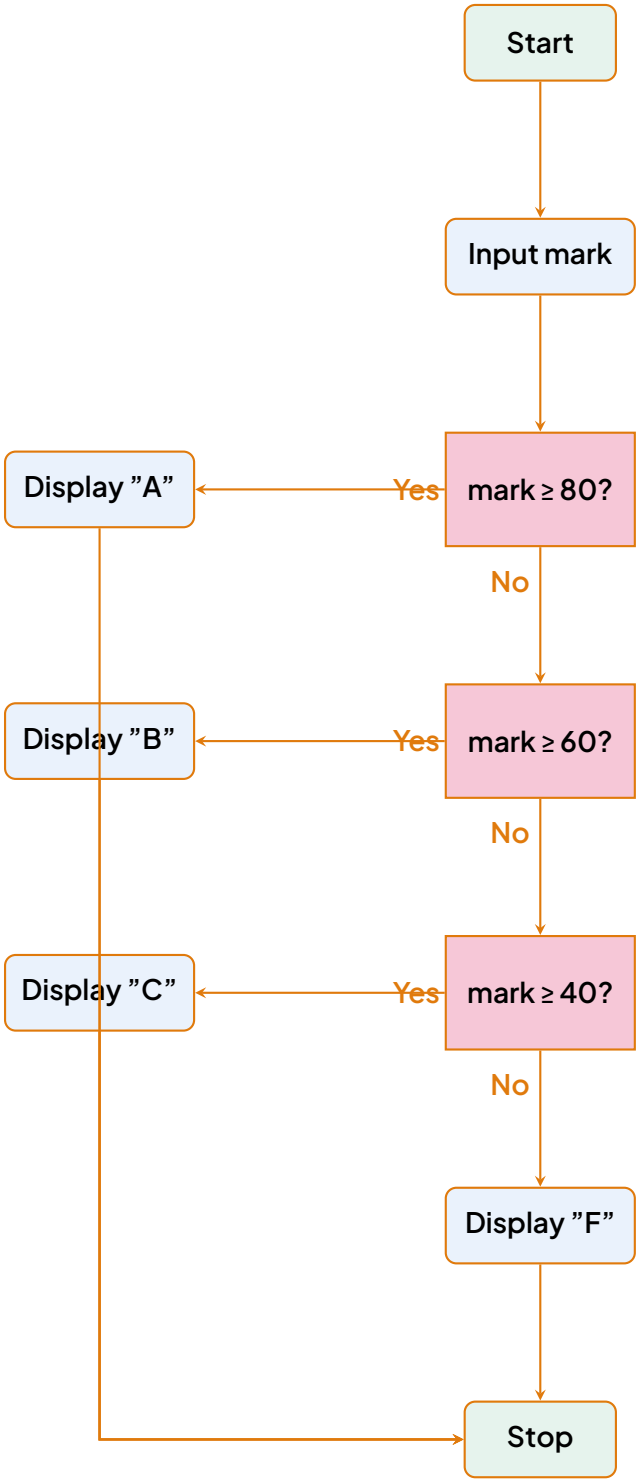
3. Level-0 DFD (showing three major processes):



Level-0 DFD for the GCE Online Registration System

SOLUTION **Exercise 7 — Completing a flowchart**

1. Correct flowchart:



Flowchart for grade determination

2. Symbol errors and corrections:

- Error 1: Using a parallelogram for decisions. Correction: Decisions must be represented by a diamond (rhombus).
- Error 2: Using a diamond for the input of the mark. Correction: Input/Output must be represented by a parallelogram.
- Error 3: Missing the `Start` symbol. Correction: Every flowchart must begin with a terminator (rounded rectangle) labelled `Start` (and end with `Stop`).

SOLUTION **Exercise 8 — Designing test cases**

The module computes average = (m1 + m2 + m3) / 3. Pass if average ≥ 10, else Fail. Each mark is out of 20.

Test #	Input data (m1, m2, m3)	Expected output	Type of test
1	(10, 10, 10)	Pass (avg = 10)	Normal (typical pass)
2	(20, 20, 20)	Pass (avg = 20)	Boundary (upper limit of marks)
3	(0, 0, 0)	Fail (avg = 0)	Boundary (lower limit of marks)
4	(9, 10, 11)	Pass (avg = 10)	Boundary (average exactly 10)
5	(9, 9, 9)	Fail (avg = 9)	Normal (typical fail)
6	(25, 10, 10)	Error / Reject	Erroneous (mark > 20)
7	(-5, 10, 10)	Error / Reject	Erroneous (mark < 0)
8	(10, 10, 9.9)	Fail (avg = 9.97)	Boundary (average just below 10)

(Any six of the above are acceptable; the table shows eight for completeness. The required six should cover normal, boundary, and erroneous cases.)

SOLUTION **Exercise 9 — Structured question: computerising a college**

1. Definition (2 marks): An information system is an organised combination of people, hardware, software, data, and procedures that collects, processes, stores, and disseminates information to support decision-making, coordination, and control in an organisation. (1 mark for key components, 1 mark for purpose.)
2. SDLC stages and activities (6 marks = 6 stages × 1 mark each):

(a) Preliminary investigation: Identify problems with paper-based results (lost scripts, slow processing). Deliverable: Project proposal.

(b) Feasibility study: Assess cost, technical requirements, staff readiness, legal aspects. Deliverable: Feasibility report.

(c) Analysis: Interview teachers, administrators, students; document current processes and requirements. Deliverable: Requirements specification (SRS).

(d) Design: Design database (students, subjects, marks), input screens for mark entry, report cards, security roles. Deliverable: System design document.

(e) Implementation: Develop/buy software, set up database, migrate past records, train staff, install hardware. Deliverable: Operational system, user manuals.

(f) Maintenance: Fix bugs, update for new grading policies, backup database, provide help-desk support. Deliverable: Maintenance logs, updated versions.

3. Changeover methods (8 marks = 4 methods × 2 marks each):
   - Direct changeover: Old system is stopped and new system starts immediately on a set date. High risk, low cost, no parallel effort.
   - Parallel running: Old and new systems operate simultaneously for a period; outputs are compared. Low risk, high cost (double workload), good for verification.
   - Phased conversion: System is implemented module by module (e.g., first term results, then annual). Risk spread out, allows gradual training.
   - Pilot conversion: New system is tried in one part of the organisation (e.g., one form/class) before full rollout. Limits risk, provides real feedback.
4. Recommendation (4 marks): Phased conversion (or pilot conversion) is most suitable. Justifications:

   (a) The school can implement one module at a time (e.g., start with Form 5 results, then Form 3, etc.), reducing the risk of total failure and allowing staff to adapt gradually.

   (b) It avoids the double workload of parallel running (teachers already overloaded) and the high risk of direct changeover (no fallback if the new system crashes during the critical result publication period).

   (Other reasonable choices with valid justifications earn marks.)

SOLUTION **Exercise 10 — Testing and IS personnel**

1. Verification vs Validation (4 marks):

- Verification: "Are we building the product right?" — checking that the mobile banking app meets its technical specifications. Example: reviewing the code of the fund-transfer module against the design document to ensure the algorithm matches the specification.
- Validation: "Are we building the right product?" — checking that the app meets the actual needs of bank customers. Example: conducting user acceptance testing with real customers performing transactions (balance check, transfer, bill payment) to see if the app is usable and fits their workflow.

(2 marks each: 1 for definition, 1 for illustration.)

2. Alpha vs Beta testing (4 marks):

- Alpha testing: Performed by internal staff (developers, QA team) at the developer's site (in Douala) in a controlled environment. Focus on finding bugs before external release.
- Beta testing: Performed by real users (selected customers) in their own environment (their phones, various networks in Cameroon). Focus on usability, compatibility, and real-world performance.

(2 marks each: 1 for who, 1 for where.)

3. Matching job titles to duties (5 marks):

Job title	Duty
Chief Information Officer (CIO)	(i) aligns IT strategy with the goals of the organisation
Systems analyst	(ii) studies user needs and specifies new systems
Network administrator	(iii) installs and maintains the network equipment
Database Administrator (DBA)	(iv) manages, secures and backs up the databases
Help-desk technician	(v) assists users with everyday technical problems

(1 mark per correct match.)

4. Maintenance classification (5 marks):

(a) Fixing a bug that doubles some transactions → Corrective maintenance (fixes an error).

(b) Adapting the application to a new version of Android → Adaptive maintenance (adapts to environment change).

(c) Adding a fingerprint login requested by users → Perfective maintenance (enhances functionality/usability).

(d) Reorganising database indexes before performance degrades → Preventive maintenance (prevents future problems).

(e) Correcting a wrong currency-conversion formula → Corrective maintenance (fixes an error in calculation).

(1 mark each.)

SOLUTION **Exercise 11 — Essay question**

Introduction The statement "A good information system is 20% technology and 80% management" highlights that the success of an information system (IS) depends far more on how it is managed, governed, and adopted than on the hardware and software alone. In the Cameroonian context — whether in schools, banks, or hospitals — many projects fail not because the technology is inadequate, but because of poor management practices. This essay discusses four critical management aspects: security, staff training, documentation, and user involvement.

Argument 1: Security as a management responsibility Technology provides firewalls, encryption, and access controls, but management defines policies, enforces compliance, and responds to incidents. For example, a bank in Douala may install the latest intrusion detection system (technology), but if management does not enforce regular password changes, conduct security audits, or train staff against phishing, the system remains vulnerable. The 2016 cyberattack on a major Cameroonian bank was largely due to management neglecting to patch known vulnerabilities and failing to implement a security incident response plan. Thus, security is predominantly a management discipline.

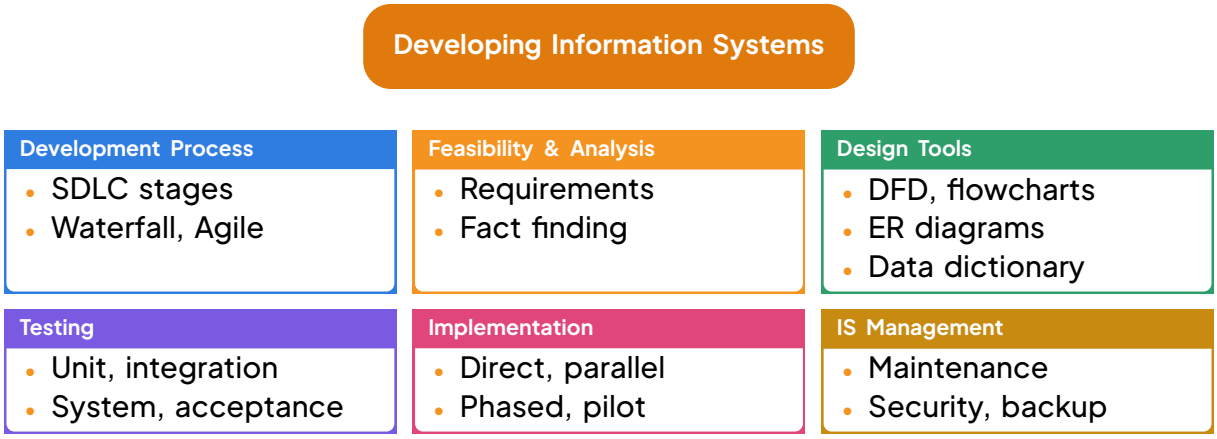
Argument 2: Staff training determines system utilisation Even the best software is useless if users cannot operate it effectively. In many Cameroonian secondary schools, computerised result management systems are installed but teachers continue using spreadsheets because they lack training. Management must allocate time and resources for continuous training, create user manuals in local languages (English/French), and establish a help desk. A hospital in Yaoundé that introduced an electronic patient record system saw adoption rise from 30% to 90% only after management mandated a two-week training programme and appointed "super-users" in each department. Training is a management decision, not a technical feature.

Argument 3: Documentation ensures continuity and maintainability Technical documentation (data dictionaries, ER diagrams, API specs) and user documentation (manuals, FAQs) are often treated as afterthoughts. However, when key developers leave — a common occurrence in Cameroon's IT sector — the system becomes unmaintainable without proper documentation. A university in Buea lost the ability to modify its student portal because the only developer who understood the codebase left and no design documents existed. Management must enforce documentation standards, conduct reviews, and archive documents as part of the SDLC. This is a governance issue, not a coding task.

Argument 4: User involvement drives relevance and acceptance Systems developed without user input often solve the wrong problems. The "waterfall" approach of gathering requirements once at the start and then disappearing until delivery leads to mismatched systems. In contrast, a microfinance institution in Bamenda adopted an agile

approach: loan officers participated in weekly demos, requested changes to the repayment schedule screen, and suggested adding USSD integration for rural clients. Management facilitated this by creating a user representative role and scheduling regular feedback sessions. The resulting system was adopted quickly because users felt ownership. User involvement is a management process of communication and collaboration. Conclusion The four arguments demonstrate that technology is merely the enabler; management provides the strategy, resources, policies, and human-centred processes that make an information system effective. In Cameroon, where resources are constrained and contexts are unique (multilingual users, unstable power, varying digital literacy), the management dimension is even more critical. Therefore, I strongly agree with the statement: a good information system is indeed 20% technology and 80% management. Organisations that invest in management practices — security governance, continuous training, rigorous documentation, and active user participation — will reap far greater returns from their IT investments than those that chase the latest technology alone.

10 SUMMARY SHEET



Mind map of the chapter "Developing information systems".

FICHE BILAN **L'essentiel à connaître pour l'examen**

1. Key definitions

- **Information system (IS)**: organised set of people, hardware, software, data and procedures that collects, processes, stores and distributes information in an organisation.
- **System development**: the complete process of analysing, designing, building, testing, implementing and maintaining an information system.
- **SDLC** (System Development Life Cycle): the structured sequence of stages through which a system passes, from initial study to maintenance.
- **Feasibility study**: investigation to decide whether a proposed system is technically, economically, operationally, legally and schedule-wise achievable.
- **Requirements specification**: precise document stating what the system must do (functional) and under what constraints (non-functional: performance, security, usability).
- **System testing**: executing the system with test data to detect errors and verify that it meets the specification.
- **Maintenance**: all modifications after delivery: corrective (fix bugs), adaptive (new environment), perfective (improvements), preventive (avoid future faults).

2. The development process (SDLC stages)

1. Preliminary study and **feasibility study**.

2. **Analysis**: fact finding (interviews, questionnaires, observation, document study) and requirements specification.
3. **Design**: general (architecture, modules) then detailed (data structures, interfaces, algorithms).
4. **Implementation / coding**: writing and documenting programs.
5. **Testing**: unit, integration, system, acceptance.
6. **Deployment**: installation, data conversion, user training.
7. **Maintenance** and evaluation.

Models to know: **waterfall** (linear, rigid), **V-model** (each stage matched by a test), **prototyping** (quick mock-up refined with users), **iterative/agile** (short cycles, user feedback).

3. System design tools

- **Flowchart**: symbols (oval = start/stop, parallelogram = input/output, rectangle = process, diamond = decision) showing the logic of a process.
- **DFD** (Data Flow Diagram): processes (circles), data stores (open rectangles), external entities (rectangles), data flows (arrows); levelled from context diagram downward.
- **Entity–Relationship diagram**: entities, attributes and relationships with cardinalities; basis of database design.
- **Data dictionary**: central catalogue defining every data item, flow, store and process.
- **Structure charts** and **pseudocode** for modular design; **Gantt chart** for project scheduling.

4. Testing and implementation

- **Unit testing**: each module tested alone. **Integration testing**: modules tested together (interfaces). **System testing**: whole system vs specification. **Acceptance (UAT)**: users validate with real data.
- **Black-box testing**: from inputs/outputs, no knowledge of code. **White-box testing**: based on internal logic and paths.
- Test data: **normal** (valid), **extreme/boundary** (limits), **erroneous** (invalid, must be rejected).
- **Alpha testing** (in-house) vs **beta testing** (selected real users).
- Changeover strategies: **direct** (old stops, new starts: cheap but risky), **parallel** (both run together: safe but costly), **phased** (module by module), **pilot** (one site first).

5. Information system management

- Roles: **CIO**, system administrator, database administrator, network administrator, support/helpdesk.
- **Security**: confidentiality, integrity, availability; controls: passwords, access rights, encryption, firewalls, antivirus, physical protection.
- **Backup** (full, incremental, differential) and **disaster recovery plan**.
- Documentation: **technical** (for developers) and **user** documentation.
- Evaluation criteria: performance, reliability, cost, user satisfaction.

6. Common mistakes to avoid

- Confusing verification (built correctly?) with validation (correct system built?).
- Mixing up unit, integration and acceptance testing, or alpha and beta testing.
- Believing direct changeover is "safe": it is the riskiest strategy.
- Drawing DFD arrows between two data stores or two entities without a process.
- Forgetting maintenance, training and documentation in the life cycle.

7. GCE tips

- Learn to define each term in one precise sentence: definitions earn easy marks.
- Be able to compare in a table: waterfall vs agile; black-box vs white-box; the four changeover strategies.
- Practise drawing a small flowchart, DFD and ER diagram with correct symbols and labels.
- For a scenario question, always justify the chosen testing type or changeover strategy with the context (cost, risk, size of organisation).

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS
OnBuch+